Nivelul transport

Săptămâna 6 — Scheme pentru învățare: protocoale de transport, TCP (Transmission Control Protocol), UDP (User Datagram Protocol), numere de port, stabilirea conexiunii în trei pași, stop-and-wait, fereastră glisantă, congestia traficului

Săptămâna 6 pe scurt: temele ca un lanț



Rolul nivelului transport	Protocoale de transport	Numere de port	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)	Mecanisme TCP (Transmission Control Protocol)	Congestia traficului
livrare de la proces la proces	familia principală de protocoale	intervale, socket-uri, 5-tuplu	antet de 8 octeți, datagrame	flux de octeți fiabil	stabilirea conexiunii, stop-and-wait, fereastră glisantă	cwnd, pornire lentă, AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă)

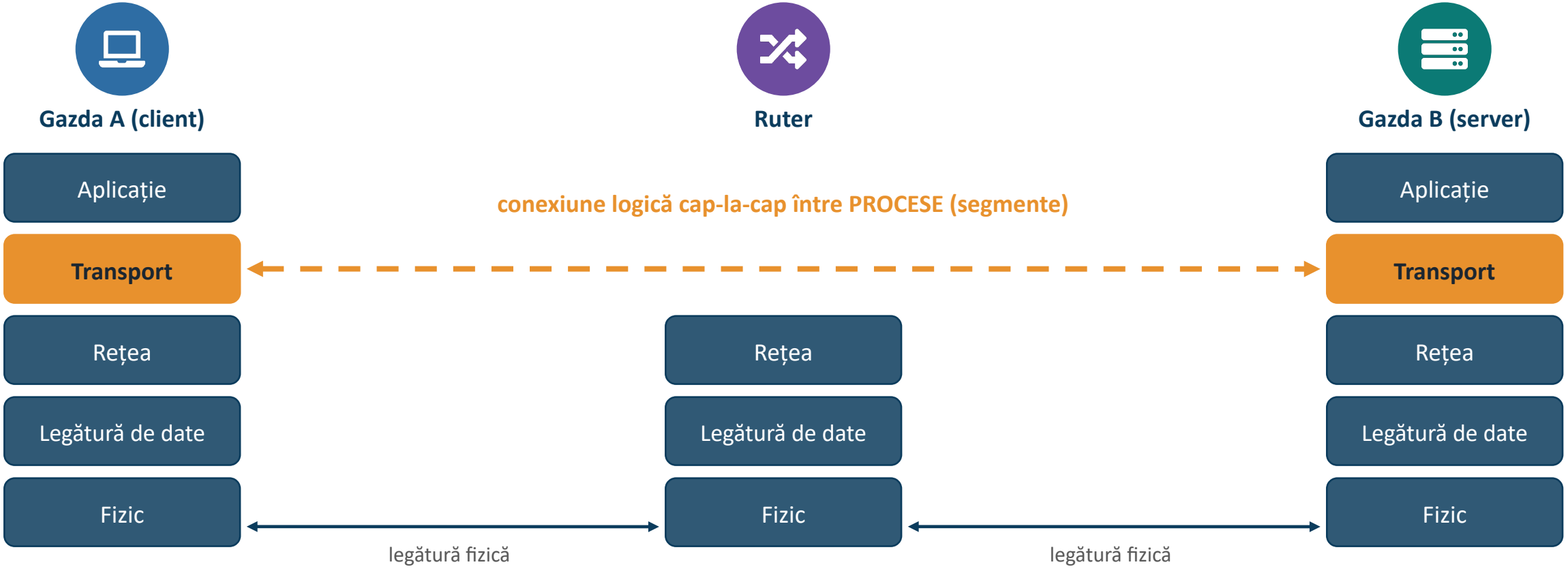
Împărțirea capitolului: Săptămâna 6 = nivelul transport propriu-zis (această prezentare). Săptămâna 7 = socket-uri și programare client-server în Java peste TCP (Transmission Control Protocol) și UDP (User Datagram Protocol), pe baza noțiunilor de mai jos.

1

Rolul nivelului transport

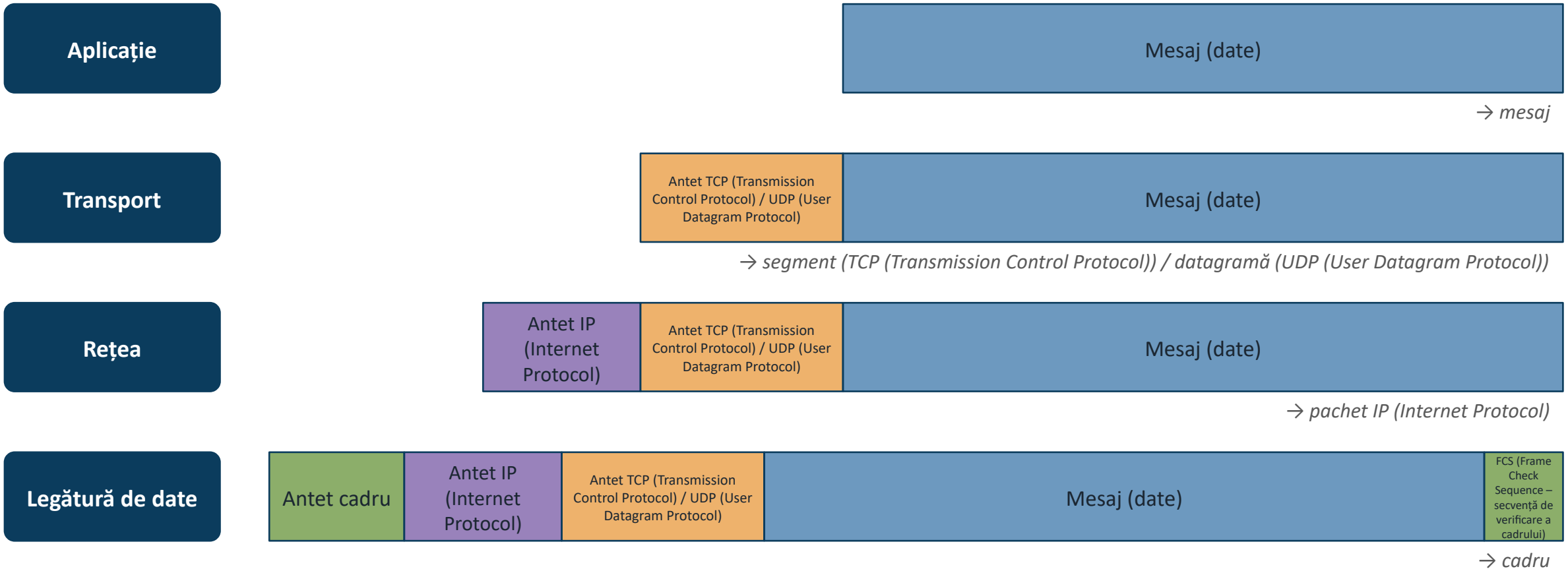
De la comunicația între gazde la comunicația între procese

Unde se află nivelul transport: doar în sistemele terminale



Ruterele se opresc la nivelul rețea: ele retransmit pachete IP (Internet Protocol) și nu inspectează niciodată starea de transport. Protocolul de transport rulează doar în gazde, de obicei în nucleul OS (Operating System – sistem de operare).

Încapsularea: unitatea de date de transport în stiva de protocoale



La coborâre, fiecare nivel își adaugă antetul (încapsulare); la urcare, fiecare nivel îl elimină (decapsulare). Antetul de transport conține numerele de PORT care identifică procesele.

Servicii pe care le poate oferi un protocol de transport — și cine le oferă

	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
Multiplexare / demultiplexare — porturile livrează datele procesului corect	✓	✓
Detecția erorilor — sumă de control peste antet + date + pseudo-antet	✓	✓
Livrare fiabilă — fără pierderi, fără duplicate, în ordine	✗	✓
Gestiunea conexiunii — stabilire și închidere explicite	✗	✓
Controlul fluxului — nu se depășește capacitatea receptorului (rwnd)	✗	✓
Controlul congestiei — nu se supraîncarcă rețeaua (cwnd)	✗	✓
Garanții de întârziere / lățime de bandă — latență limitată, rată rezervată	✗	✗

Suma de control: opțională în UDP (User Datagram Protocol) peste IPv4 (Internet Protocol versiunea 4), obligatorie în UDP (User Datagram Protocol) peste IPv6 (Internet Protocol versiunea 6) și întotdeauna în TCP (Transmission Control Protocol). În Internetul public nimeni nu garantează întârzierea sau lățimea de bandă.

2

Protocoale de transport

UDP (User Datagram Protocol), TCP (Transmission Control Protocol) și alternativele specializate

Familia protocoalelor de transport

Protocoale de transport (TCP/IP (Transmission Control Protocol/Internet Protocol – suita de protocoale TCP/IP))

UDP (User Datagram Protocol)

RFC (Request for Comments – document de standardizare IETF) 768 · 1980

- fără conexiune
- orientat pe mesaje
- fără fiabilitate
- antet de 8 octeți

Folosit de: DNS (Domain Name System – sistemul numelor de domeniu), DHCP (Dynamic Host Configuration Protocol), NTP (Network Time Protocol), SNMP (Simple Network Management Protocol), VoIP (Voice over Internet Protocol – voce prin IP), jocuri

TCP (Transmission Control Protocol)

RFC (Request for Comments – document de standardizare IETF) 793 → 9293

- orientat pe conexiune
- flux de octeți fiabil
- control al fluxului + al congestiei
- antet de 20–60 de octeți

Folosit de: Web, e-mail, SSH (Secure Shell), FTP (File Transfer Protocol), baze de date

SCTP (Stream Control Transmission Protocol)

RFC (Request for Comments – document de standardizare IETF) 9260

- mesaje fiabile
- fluxuri multiple
- multi-homing

Folosit de: semnalizare telecom, date WebRTC (Web Real-Time Communication – comunicație web în timp real)

DCCP (Datagram Congestion Control Protocol)

RFC (Request for Comments – document de standardizare IETF) 4340

- nefiabil
- dar cu control al congestiei

Folosit de: streaming media experimental

QUIC (Quick UDP Internet Connections)

RFC (Request for Comments – document de standardizare IETF) 9000 · peste UDP (User Datagram Protocol)

- criptat (TLS (Transport Layer Security) 1.3)
- fluxuri, fără blocare HOL (Head-Of-Line – blocare la capul cozii)
- stabilire rapidă în 0/1-RTT (Round-Trip Time – timp dus-întors)

Folosit de: HTTP/3 (Hypertext Transfer Protocol versiunea 3)

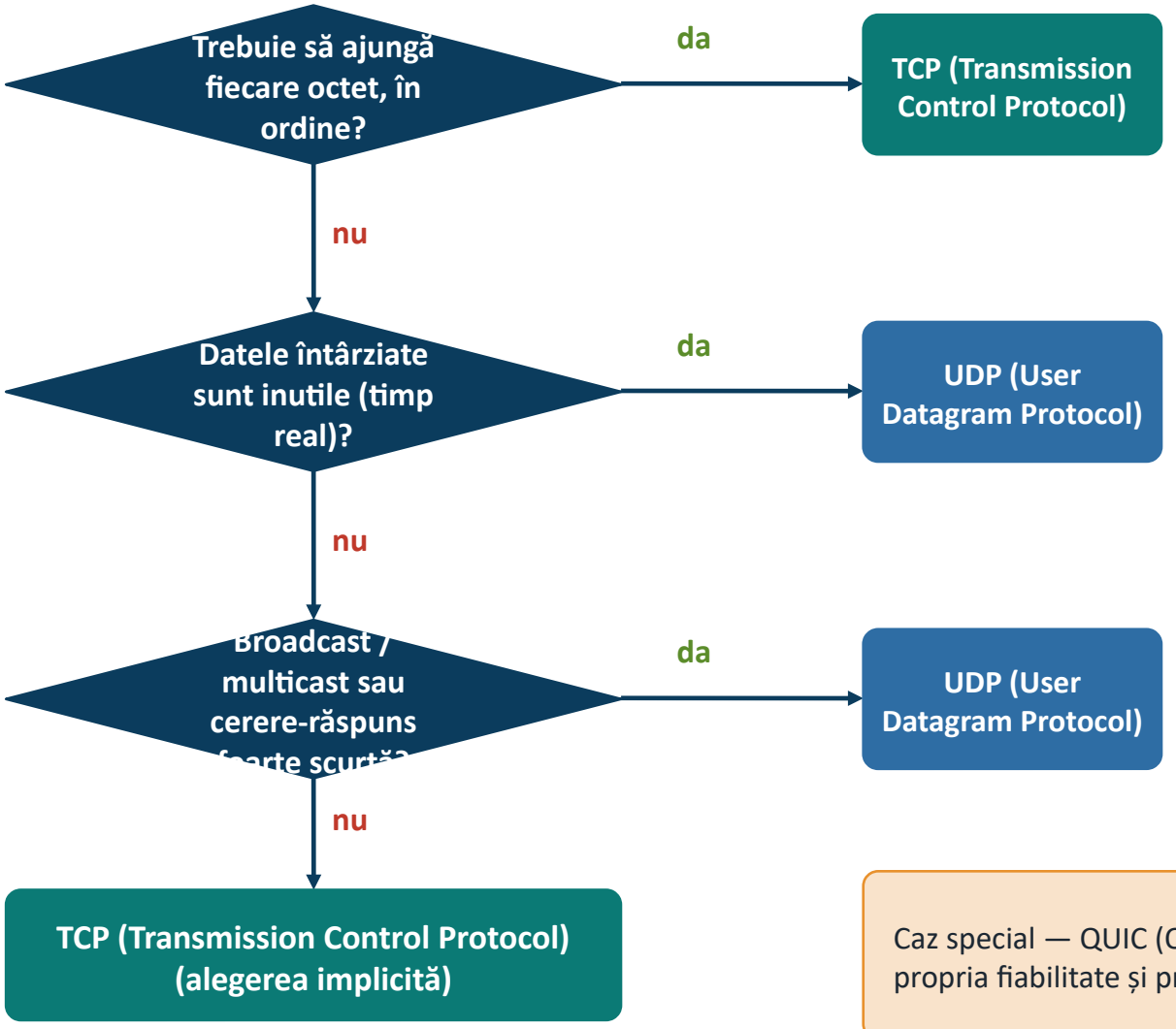
UDP (User Datagram Protocol) și TCP (Transmission Control Protocol) transportă aproape tot traficul Internet; QUIC (Quick UDP Internet Connections) este un protocol de transport implementat în spațiul utilizator, peste UDP (User Datagram Protocol).

TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol)

	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
Conexiune	fără conexiune — doar se trimite	stabilire în trei pași înaintea datelor
Unitatea de date	datagramă, limitele sunt păstrate	flux de octeți, fără limite
Fiabilitate	niciuna: pierderi, duplicate, reordonare posibile	ACKs (Acknowledgments – confirmări) + retransmisii + numere de secvență
Ordonare	negarantată	garantată
Controlul fluxului	nu	da — fereastra de recepție (rwnd)
Controlul congestiei	nu	da — fereastra de congestie (cwnd)
Dimensiunea antetului	8 octeți	20 de octeți (până la 60 cu opțiuni)
Parteneri	1-la-1, broadcast, multicast	exact 2 capete, full-duplex
Starea serverului	niciuna pentru fiecare client	o conexiune (TCB (Transmission Control Block – bloc de control al conexiunii)) pentru fiecare client

Regulă practică: UDP (User Datagram Protocol) = viteză și simplitate, aplicația tratează problemele; TCP (Transmission Control Protocol) = corectitudine, OS (Operating System – sistem de operare) tratează problemele.

Schemă de decizie: TCP (Transmission Control Protocol) sau UDP (User Datagram Protocol)?



pagini web, transfer de fișiere, e-mail, sesiuni la distanță, baze de date

apeluri voce/video, streaming live, jocuri online — datele retransmise ar ajunge prea târziu

descoperire DHCP (Dynamic Host Configuration Protocol), interogări DNS (Domain Name System – sistemul numelor de domeniu), NTP (Network Time Protocol), SNMP (Simple Network Management Protocol), descoperirea serviciilor

Caz special — QUIC (Quick UDP Internet Connections): vrea fiabilitate ȘI latență mică → își construiește propria fiabilitate și propriul control al congestiei peste UDP (User Datagram Protocol)

3

Numere de port, socket-uri și multiplexare

Identificatori pe 16 biți care numesc procesele comunicante

Spațiul numerelor de port: 16 biți, trei intervale

Port = întreg fără semn pe 16 biți → 0 … 65535. TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) au spații de porturi SEPARATE (TCP (Transmission Control Protocol) 53 ≠ UDP (User Datagram Protocol) 53).



↑ 0–1023 (doar 1,6 % din spațiu)

Porturi binecunoscute (de sistem) (0 – 1023)

- alocate de IANA (Internet Assigned Numbers Authority – autoritatea de alocare a numerelor Internet) serviciilor standard
- Linux/Unix: asocierea necesită root
- de ex. 22 SSH (Secure Shell), 53 DNS (Domain Name System – sistemul numelor de domeniu), 80 HTTP (Hypertext Transfer Protocol), 443 HTTPS (Hypertext Transfer Protocol Secure)

Porturi înregistrate (de utilizator) (1024 – 49151)

- înregistrate la IANA (Internet Assigned Numbers Authority – autoritatea de alocare a numerelor Internet) pentru fiecare aplicație
- nu necesită drepturi speciale
- de ex. 3306 MySQL, 3389 RDP (Remote Desktop Protocol), 5432 PostgreSQL, 8080

Porturi dinamice / efemere (49152 – 65535 (IANA (Internet Assigned Numbers Authority – autoritatea de alocare a numerelor Internet)))

- nu se alocă niciodată serviciilor
- alese de OS (Operating System – sistem de operare) pentru socket-urile CLIENT
- Windows: 49152–65535 · Linux: 32768–60999

Verificați pe calculatorul vostru:
Windows → `netsh int ipv4 show dynamicport tcp` Linux → `sysctl net.ipv4.ip_local_port_range`

Porturi binecunoscute de reținut

20/21	**FTP (File Transfer Protocol) date/control** TCP (Transmission Control Protocol)
22	**SSH (Secure Shell)** TCP (Transmission Control Protocol)
23	**Telnet** TCP (Transmission Control Protocol)
25	**SMTP (Simple Mail Transfer Protocol)** TCP (Transmission Control Protocol)
53	**DNS (Domain Name System – sistemul numelor de domeniu)** UDP (User Datagram Protocol) + TCP (Transmission Control Protocol)
67/68	**DHCP (Dynamic Host Configuration Protocol) server/client** UDP (User Datagram Protocol)
69	**TFTP (Trivial File Transfer Protocol)** UDP (User Datagram Protocol)
80	**HTTP (Hypertext Transfer Protocol)** TCP (Transmission Control Protocol)
110	**POP3 (Post Office Protocol versiunea 3)** TCP (Transmission Control Protocol)
123	**NTP (Network Time Protocol)** UDP (User Datagram Protocol)
143	**IMAP (Internet Message Access Protocol)** TCP (Transmission Control Protocol)
161/162	**SNMP (Simple Network Management Protocol) / trap-uri** UDP (User Datagram Protocol)
443	**HTTPS (Hypertext Transfer Protocol Secure)** TCP (Transmission Control Protocol) + UDP (User Datagram Protocol)
3306	**MySQL** TCP (Transmission Control Protocol)
3389	**RDP (Remote Desktop Protocol)** TCP (Transmission Control Protocol) + UDP (User Datagram Protocol)
8080	**HTTP (Hypertext Transfer Protocol) alternativ** TCP (Transmission Control Protocol)

Cod de culori: verde-albastru = TCP (Transmission Control Protocol), albastru = UDP (User Datagram Protocol), mov = ambele. Lista completă: /etc/services (Linux) sau C:\Windows\System32\drivers\etc\services.

Adresa de socket și 5-tuplul

Adresă de socket = (adresă IP (Internet Protocol), port)

`192.168.1.10` : `51544`

identifică GAZDA (nivelul rețea) — identifică PROCESUL

Conexiune TCP (Transmission Control Protocol) = 5-tuplu

protocol	`TCP (Transmission Control Protocol)`
IP (Internet Protocol) sursă	`192.168.1.10`
port sursă	`51544`
IP (Internet Protocol) destinație	`93.184.215.14`
port destinație	`443`

Exemplu — două file către același server web

- Fila 1: (TCP (Transmission Control Protocol), 192.168.1.10, 51544, 93.184.215.14, 443) Fila 2: (TCP (Transmission Control Protocol), 192.168.1.10, 51545, 93.184.215.14, 443)
- Același socket destinație, port de client diferit → două conexiuni diferite. Un server poate deci servi mii de clienți pe UN SINGUR port.

Demultiplexarea: cum își găsește socket-ul un segment sosit

TCP (Transmission Control Protocol) B:50001 → A:80

→ conex. ↔ B:50001

TCP (Transmission Control Protocol) C:50001 → A:80

→ conex. ↔ C:50001 (alt IP (Internet Protocol) sursă)

TCP (Transmission Control Protocol) D:40000 → A:80 SYN (Synchronize – sincronizare)

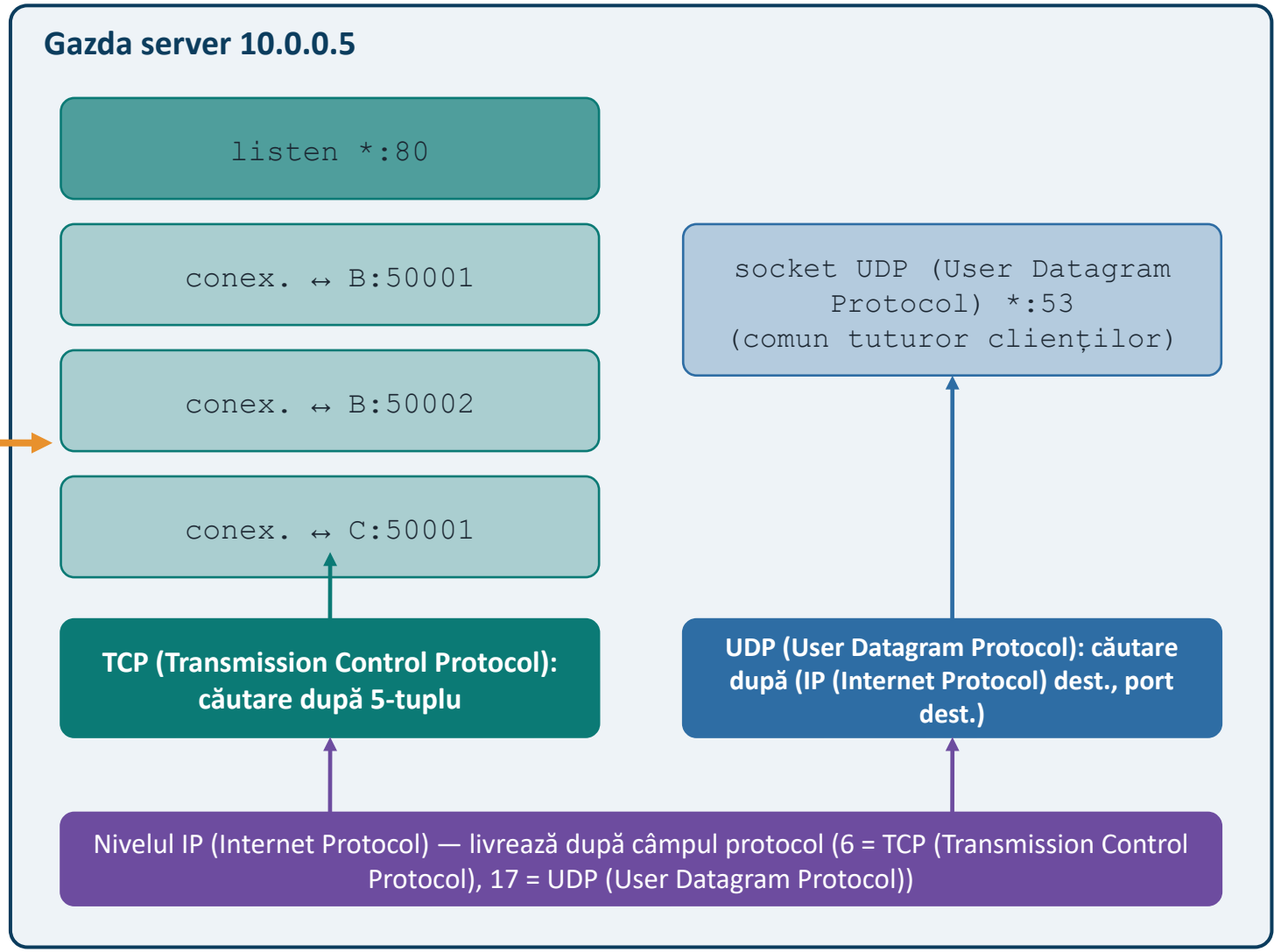
→ socket de ascultare (conexiune nouă)

UDP (User Datagram Protocol) B:50001 → A:53

→ socket UDP (User Datagram Protocol) *:53

UDP (User Datagram Protocol) C:61000 → A:53

→ același socket UDP (User Datagram Protocol) *:53

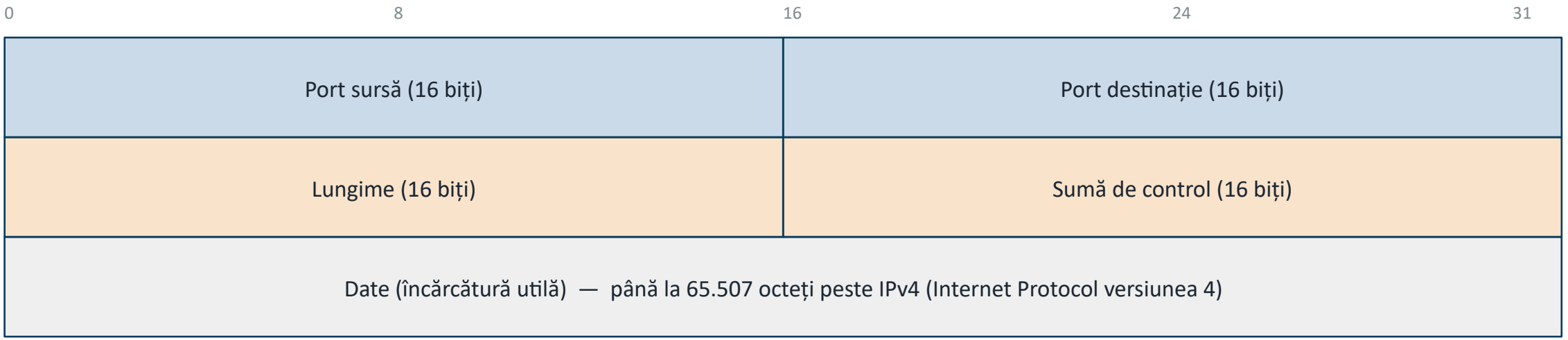


4

UDP (User Datagram Protocol)

Transportul minimal: porturi + sumă de control peste IP (Internet Protocol)

Antetul UDP (User Datagram Protocol): 4 câmpuri, 8 octeți



Port sursă — portul emițătorului; poate fi 0 când nu se așteaptă răspuns

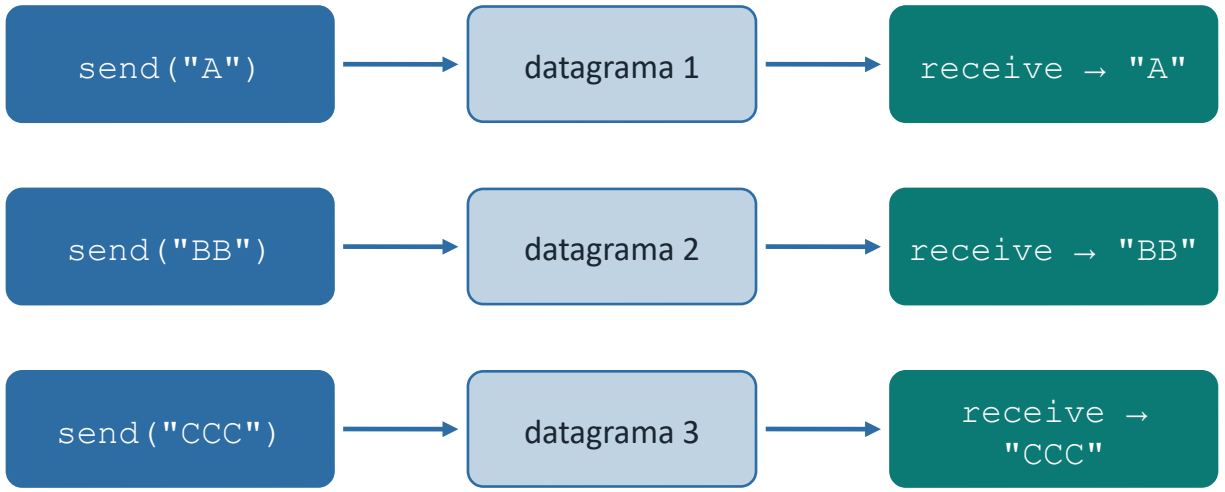
Port destinație — portul procesului receptor — cheia de demultiplexare

Lungime — antet + date, în octeți (minimum 8)

Sumă de control — complement față de unu pe 16 biți peste pseudo-antet + antet + date; 0 = necalculată (doar IPv4 (Internet Protocol versiunea 4))

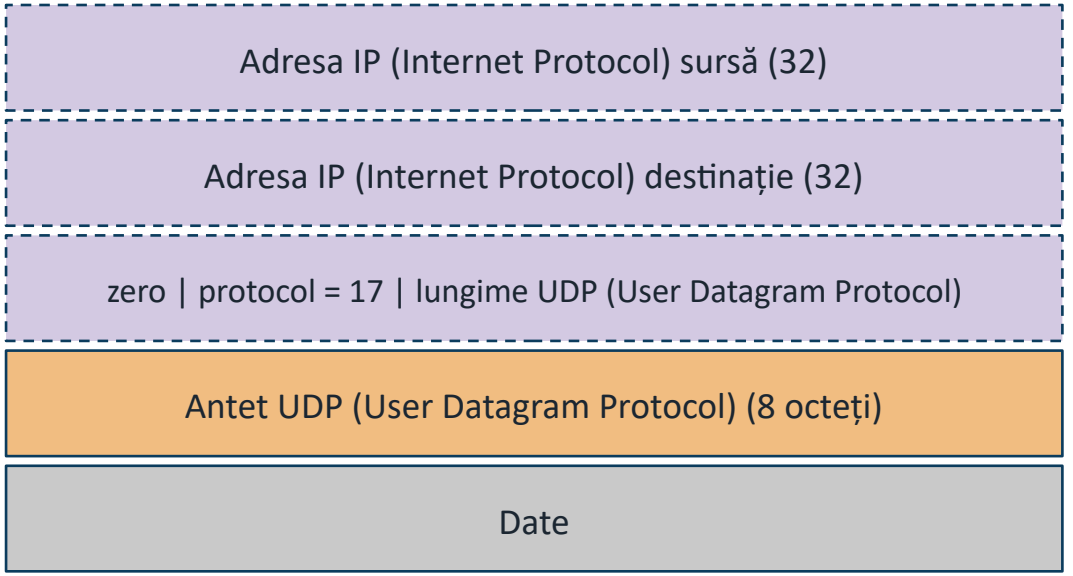
Comportamentul UDP (User Datagram Protocol): un send = o datagramă = un receive

Limitele mesajelor sunt păstrate



…dar oricare datagramă se poate pierde, duplica sau poate sosi în altă ordine — UDP (User Datagram Protocol) nu observă.

Acoperirea sumei de control (pseudo-antet)



Linie întreruptă = pseudo-antet: preluat din IP (Internet Protocol) doar pentru calcul, nu se transmite niciodată. Detectează datagramele livrate greșit.

Avantaje

- fără întârziere de stabilire · antet de 8 octeți · fără stare per client
- aplicația decide exact ce și când trimite

Dezavantaje

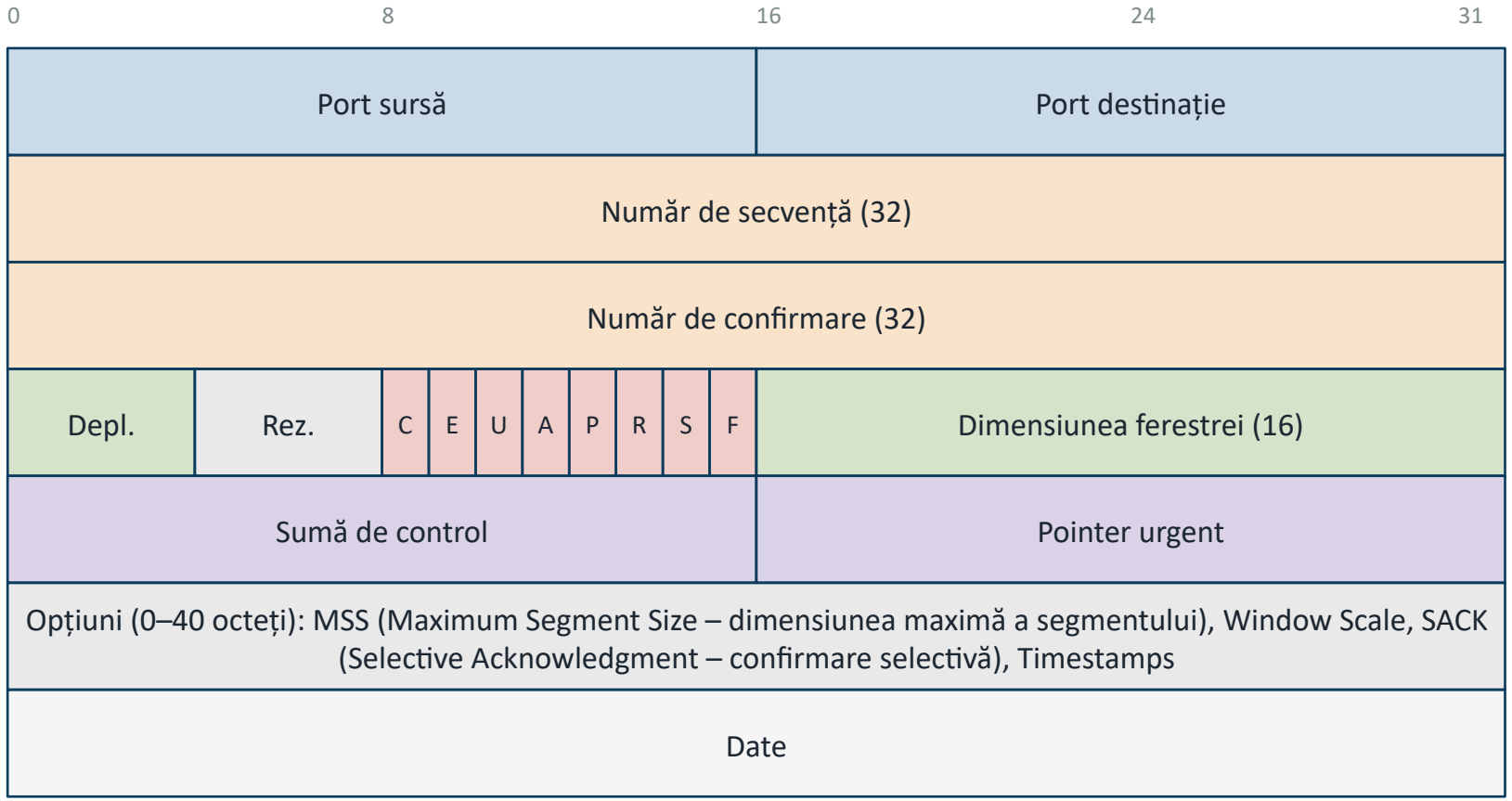
- fără fiabilitate, ordonare sau control al fluxului
- fără control al congestiei — poate afecta alte fluxuri

5

TCP (Transmission Control Protocol)

Flux de octeți fiabil, ordonat, full-duplex

Antetul TCP (Transmission Control Protocol): 20 de octeți + până la 40 de octeți de opțiuni



Porturi — identifică cele două socket-uri

Nr. de secvență — numărul primului OCTET DE DATE din segment (ISN (Initial Sequence Number – număr de secvență inițial) într-un SYN (Synchronize – sincronizare))

Nr. de confirmare — următorul octet așteptat de la celălalt capăt (cumulativ)

Deplasament date — lungimea antetului în cuvinte de 32 de biți (5–15)

Indicatori — CWR (Congestion Window Reduced – fereastră de congestie redusă) ECE (ECN-Echo – ecou al notificării explicite de congestie) URG (Urgent – urgent) ACK (Acknowledgment – confirmare) PSH (Push – livrare imediată) RST (Reset – resetare) SYN (Synchronize – sincronizare) FIN (Finish – terminare)

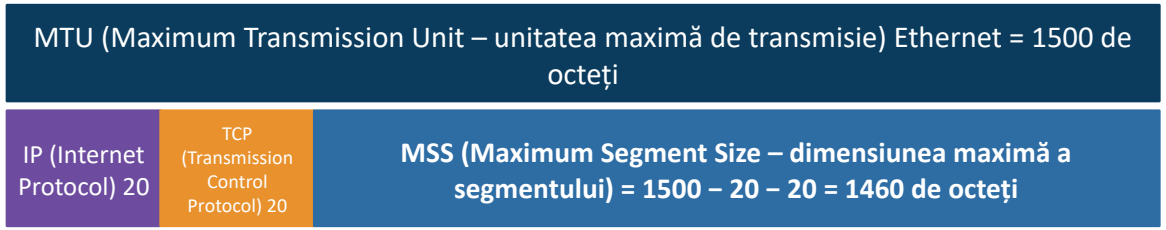
Fereastră — spațiul liber din bufferul de recepție (rwnd)

Sumă de control — pseudo-antet + antet + date

Indicatorii TCP (Transmission Control Protocol), opțiunile și MSS (Maximum Segment Size – dimensiunea maximă a segmentului)

SYN (Synchronize – sincronizare)	sincronizează numerele de secvență — deschidere
ACK (Acknowledgment – confirmare)	câmpul de confirmare este valid
FIN (Finish – terminare)	emițătorul a terminat — închide un sens
RST (Reset – resetare)	resetare — abandonează conexiunea
PSH (Push – livrare imediată)	livrează imediat datele aplicației
URG (Urgent – urgent)	pointerul urgent este valid (rar)
ECE (ECN-Echo – ecou al notificării explicite de congestie)	ecou ECN (Explicit Congestion Notification – notificare explicită a congestiei) — s-a constatat congestie
(Congestion Window Reduced – fereastră de congestie	fereastra de congestie a fost redusă

MSS (Maximum Segment Size – dimensiunea maximă a segmentului) (date pe segment)



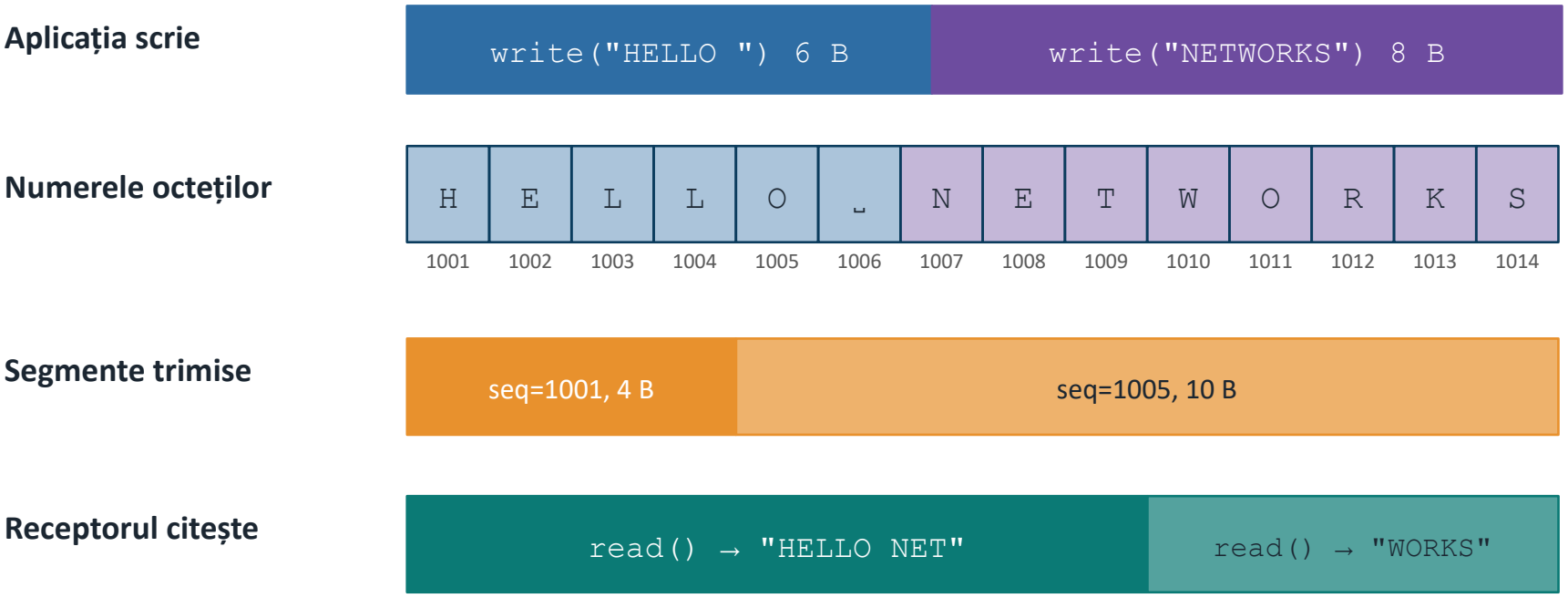
Fiecare parte își anunță MSS (Maximum Segment Size – dimensiunea maximă a segmentului) în SYN (Synchronize – sincronizare); se folosește valoarea mai mică.

Opțiuni uzuale (în segmentele SYN (Synchronize – sincronizare))

- MSS (Maximum Segment Size – dimensiunea maximă a segmentului) — cel mai mare segment pe care îl poate primi emițătorul
- Window Scale (WS (Window Scale – scalarea ferestrei)) — rwnd × 2^S, S ≤ 14 → ferestre de până la ≈ 1 GiB (gibioctet)
- SACK (Selective Acknowledgment – confirmare selectivă)-permitted / blocuri SACK (Selective Acknowledgment – confirmare selectivă) — raportează datele primite necontiguu
- Timestamps — măsurarea RTT (Round-Trip Time – timp dus-întors), protecție la reluarea ciclică a numerelor de secvență

TCP (Transmission Control Protocol) este un flux de octeți: se numerotează octeți, nu mesaje

Aplicația scrie octeți; TCP (Transmission Control Protocol) taie fluxul în segmente; receptorul citește octeți — limitele write/read NU se păstrează.



Consecințe

- seq = numărul primului octet din segment
- ack = următorul octet așteptat
- aplicația trebuie să-și delimiteze mesajele (delimitator sau lungime)

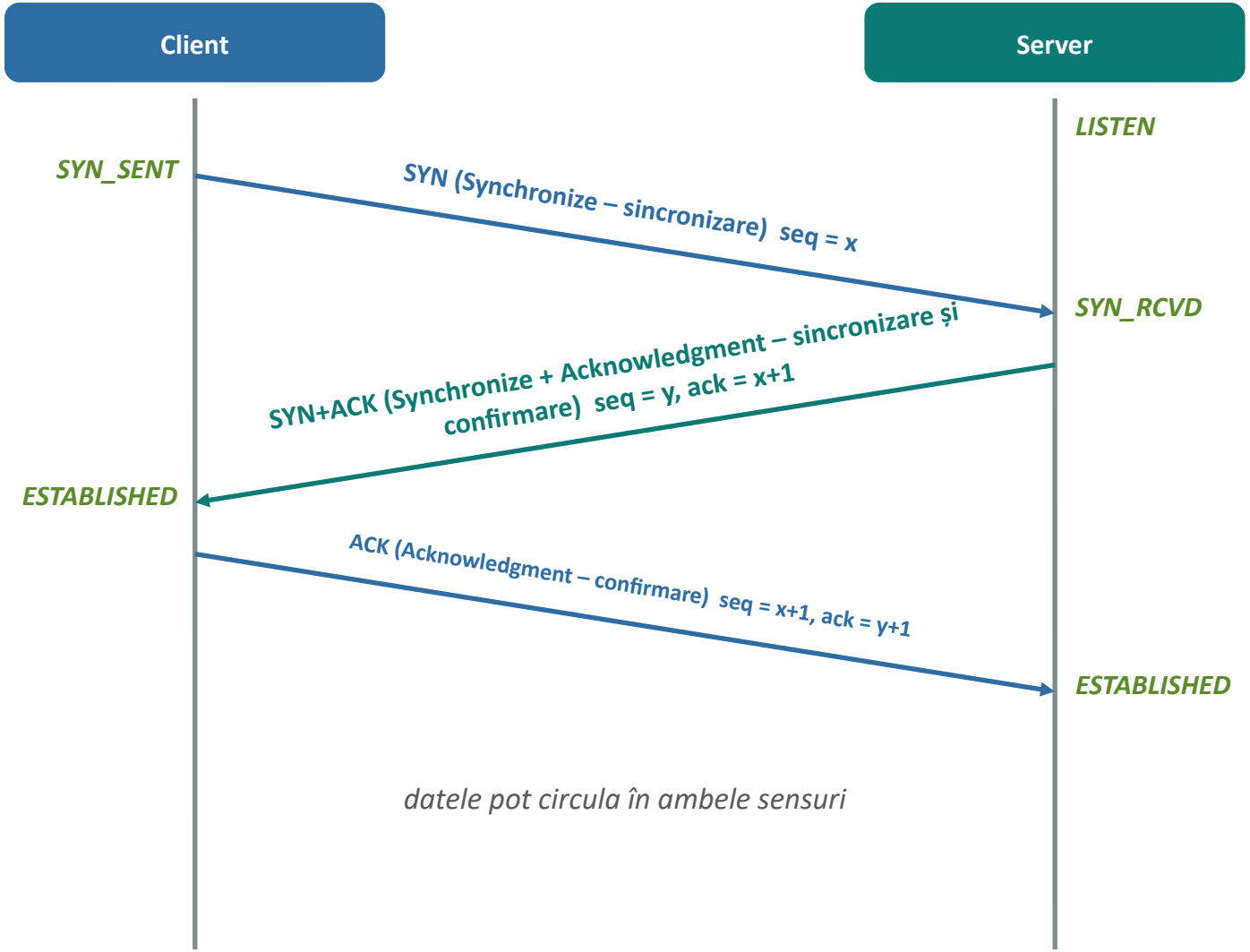
Cinci proprietăți ale TCP (Transmission Control Protocol): orientat pe conexiune · punct-la-punct și full-duplex · fiabil (seq, ACK (Acknowledgment – confirmare), sumă de control, retransmisie) · controlul fluxului (rwnd) · controlul congestiei (cwnd)

6

Mecanisme TCP (Transmission Control Protocol)

Stabilirea în trei pași · închiderea · stop-and-wait · fereastra glisantă · temporizatoare

Stabilirea în trei pași: deschiderea unei conexiuni TCP (Transmission Control Protocol)



Ce face fiecare segment

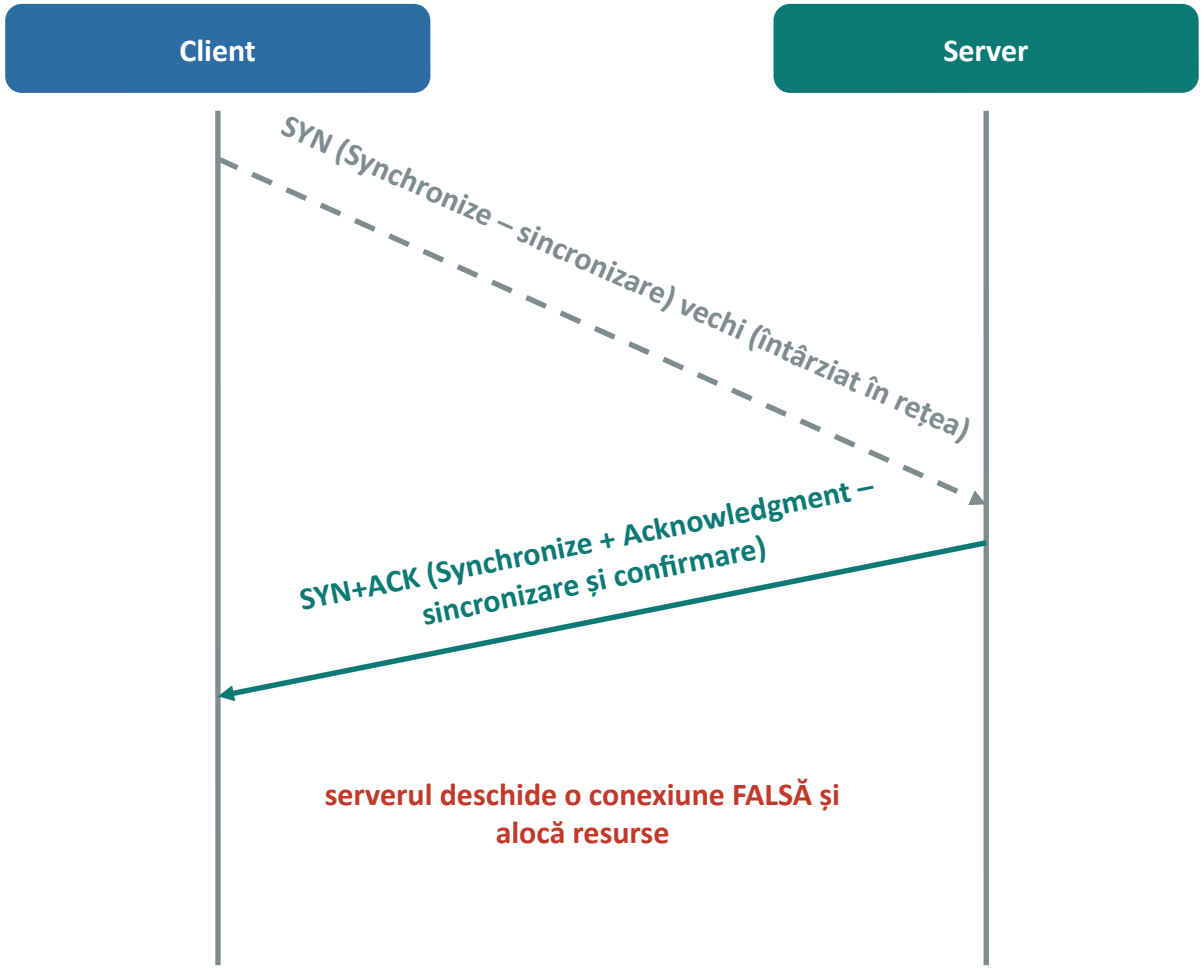
- 1. clientul își propune ISN (Initial Sequence Number – număr de secvență inițial) x
- 2. serverul acceptă x (ack = x+1) și își propune ISN (Initial Sequence Number – număr de secvență inițial) y
- 3. clientul acceptă y (ack = y+1)
- SYN (Synchronize – sincronizare) consumă un număr de secvență → +1

Negociate tot în SYNs (Synchronize – segmente de sincronizare)

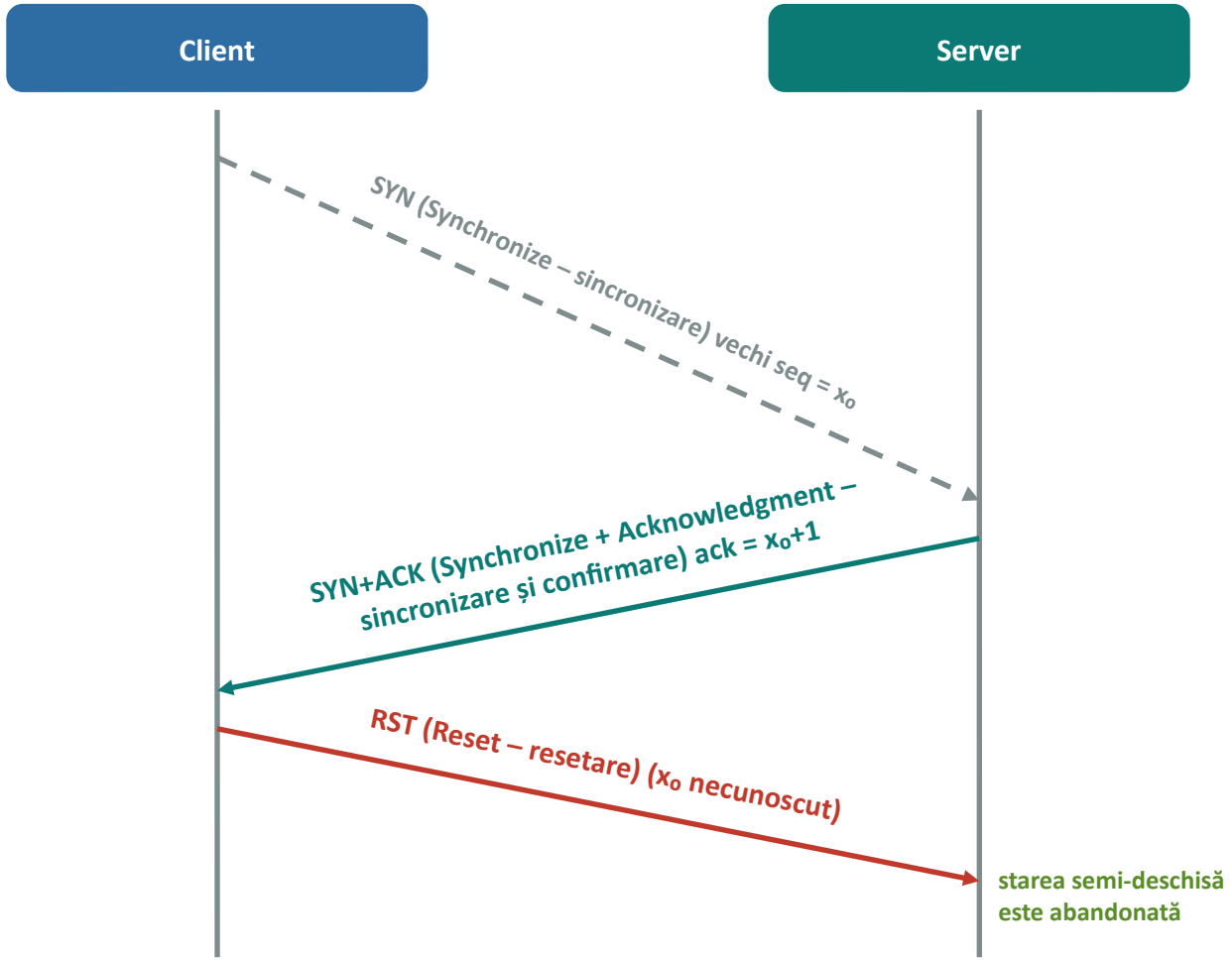
- MSS (Maximum Segment Size – dimensiunea maximă a segmentului) al fiecărei părți
- Factorul Window Scale
- SACK (Selective Acknowledgment – confirmare selectivă) permitted, Timestamps
- ISN (Initial Sequence Number – număr de secvență inițial) ales pseudo-aleator (RFC (Request for Comments – document de standardizare IETF) 6528): duplicatele vechi sunt respinse, numerele de secvență sunt greu de ghicit

De ce trei segmente? Problema unui SYN (Synchronize – sincronizare) duplicat întârziat

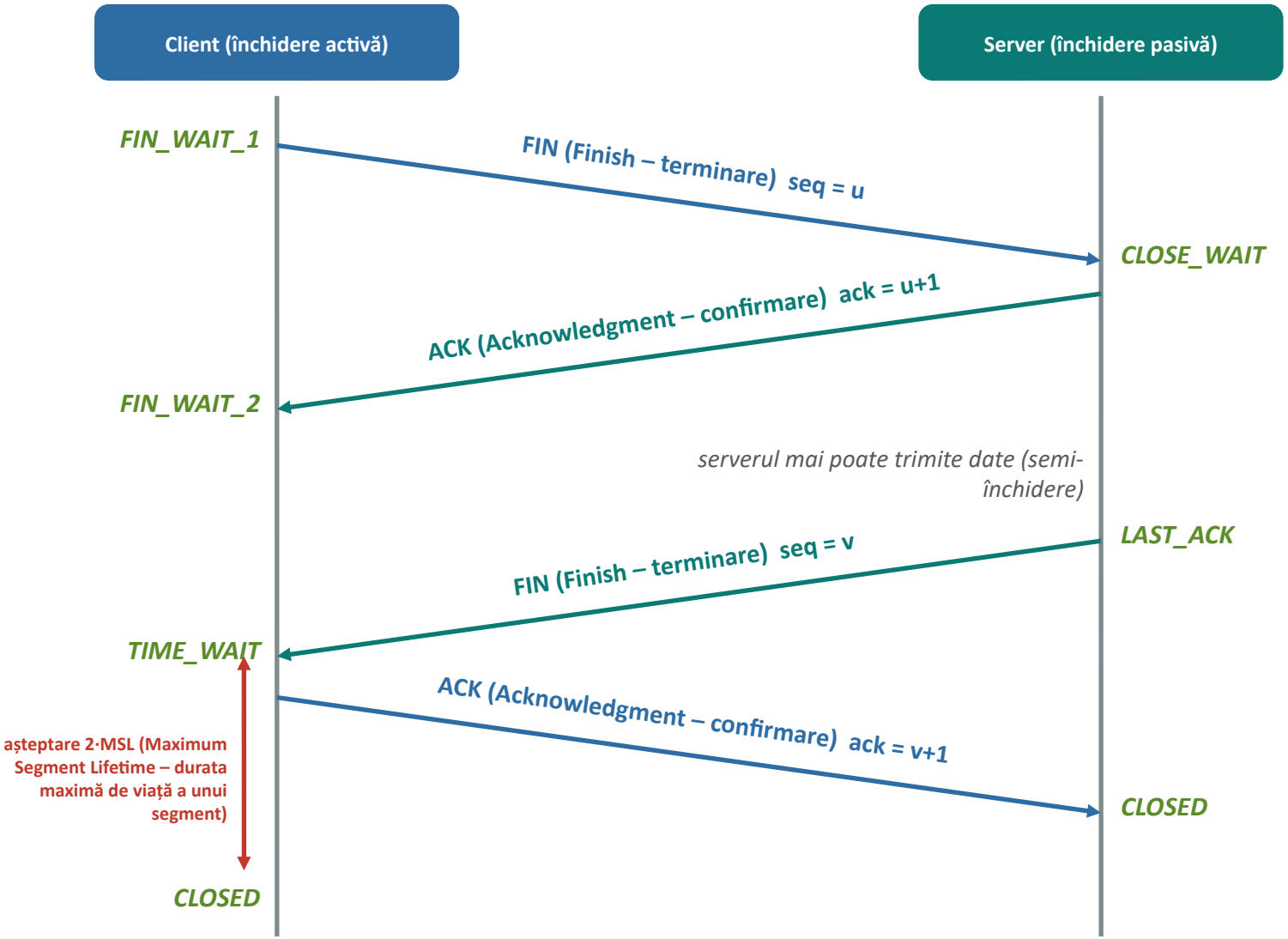
Cu doar 2 segmente (SYN (Synchronize – sincronizare), SYN+ACK (Synchronize + Acknowledgment – sincronizare și confirmare))…



…cu 3 segmente clientul poate refuza



Închiderea conexiunii: câte un FIN (Finish – terminare) în fiecare sens



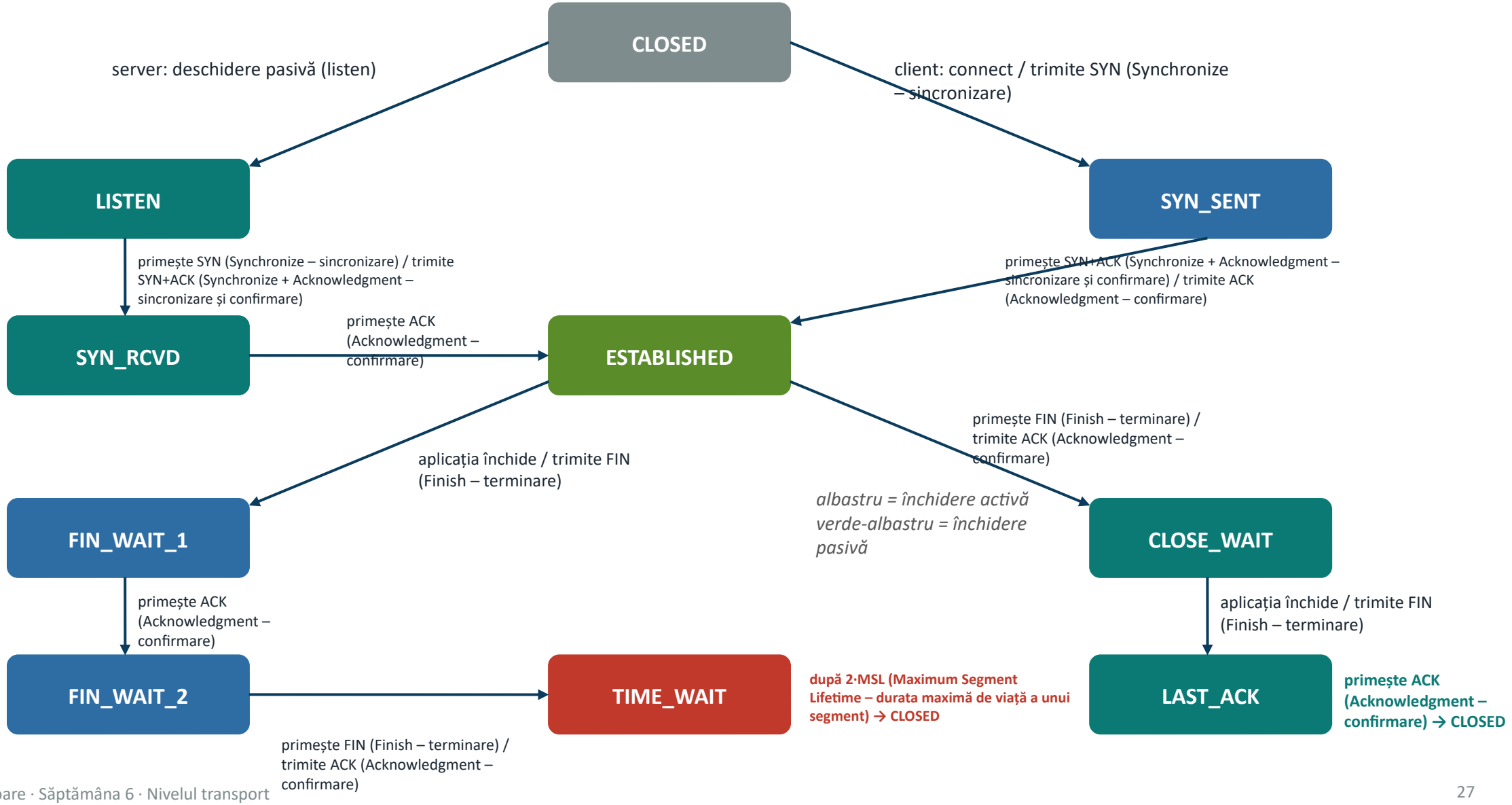
De ce TIME_WAIT (2·MSL (Maximum Segment Lifetime – durata maximă de viață a unui segment))?

- ultimul ACK (Acknowledgment – confirmare) se poate pierde → partenerul retrimite FIN (Finish – terminare), iar noi trebuie să răspundem
- duplicatele vechi ale acestui 5-tuplu trebuie să dispară înainte de refolosirea lui
- Linux: 60 s; Windows: implicit 120 s

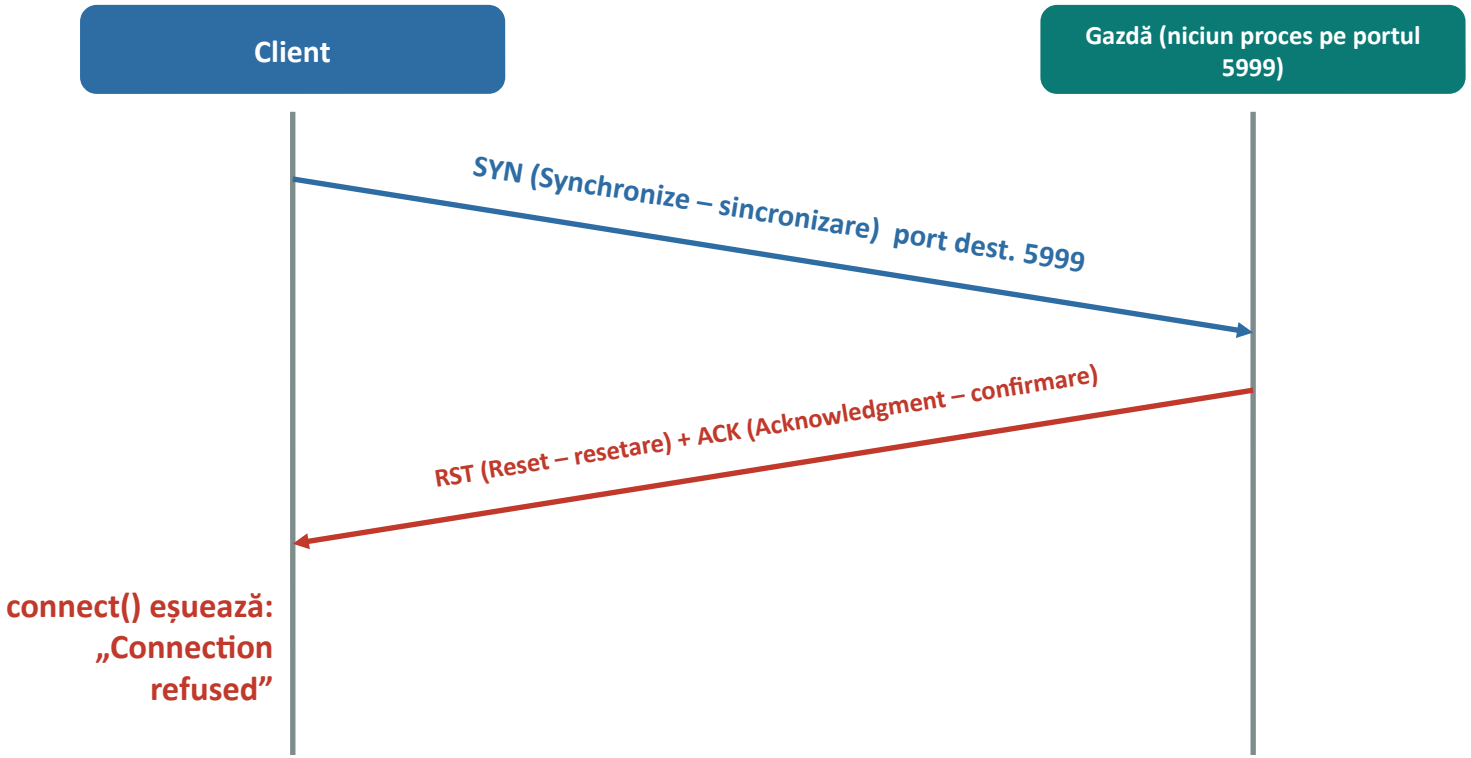
Variante

- 3 segmente: serverul combină ACK (Acknowledgment – confirmare) + FIN (Finish – terminare)
- închidere simultană: ambele părți trimit FIN (Finish – terminare)
- abandon: RST (Reset – resetare) — fără procedură de închidere, datele se pierd
- FIN (Finish – terminare) consumă un număr de secvență

Stările unei conexiuni TCP (Transmission Control Protocol) (automat simplificat)



RST (Reset – resetare): când TCP (Transmission Control Protocol) spune „nu” — conectarea la un port închis

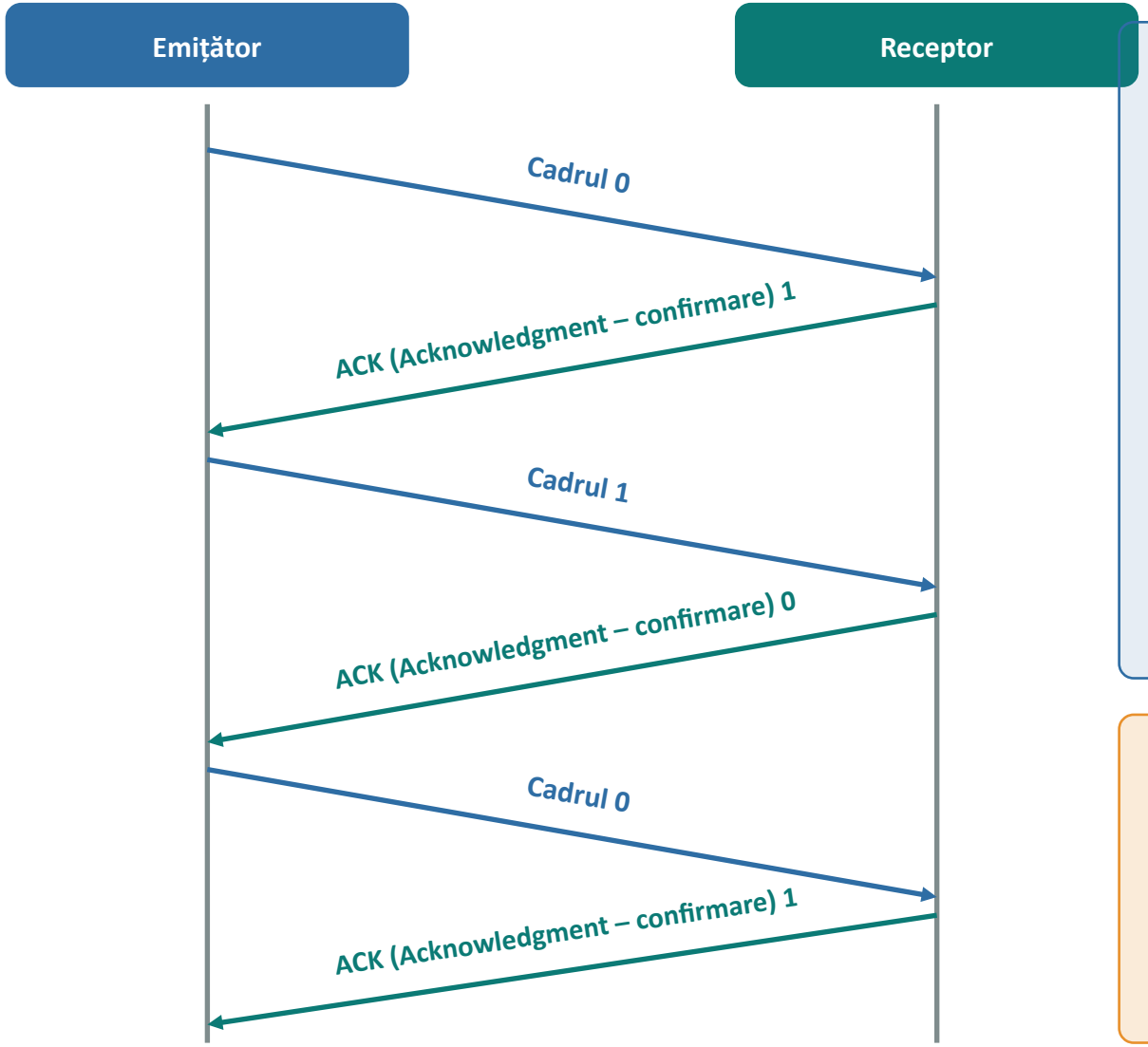


RST (Reset – resetare) se trimite și când un proces se oprește cu o conexiune deschisă sau când sosește un segment pentru o conexiune inexistentă.

Trei rezultate ale unei încercări de conectare

- **Port deschis → SYN+ACK (Synchronize + Acknowledgment – sincronizare și confirmare)**
  - stabilirea conexiunii reușește
- **Port închis → RST (Reset – resetare)**
  - eșec imediat: Connection refused
- **Port filtrat (aruncat de firewall) → nimic**
  - SYN (Synchronize – sincronizare) retransmis cu temporizare crescătoare, apoi expirare
- Instrumente: Test-NetConnection (Windows), nc -zv (Linux)

ARQ (Automatic Repeat reQuest – cerere automată de repetare) stop-and-wait: cel mai simplu protocol fiabil



Emițător

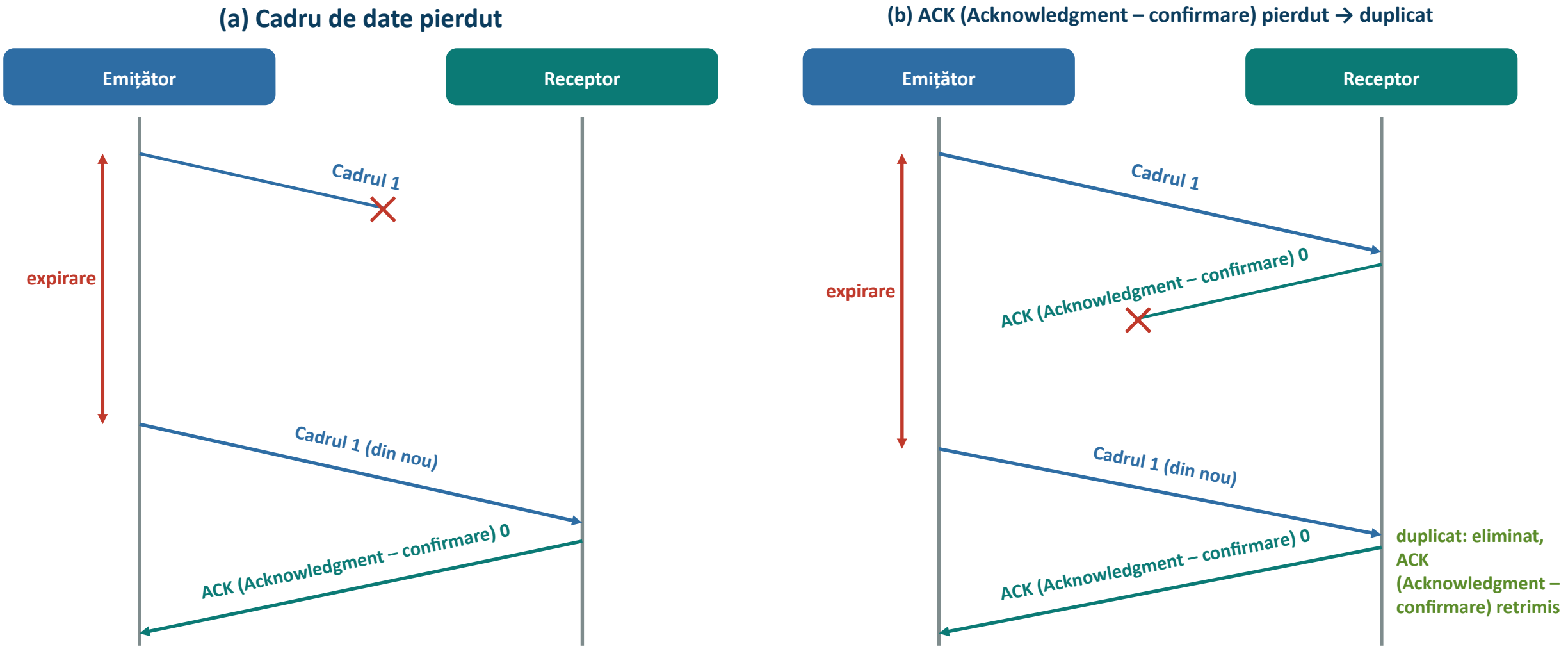
- trimite cadrul, pornește temporizatorul
- așteaptă ACK (Acknowledgment – confirmare)
- sosește ACK (Acknowledgment – confirmare) → cadrul următor
- expiră temporizatorul → retrimite același cadru

Receptor

- verifică suma de control
- cadru nou → îl livrează, trimite ACK (Acknowledgment – confirmare)
- duplicat → îl elimină, retrimite ACK (Acknowledgment – confirmare)
- corupt → îl elimină

Număr de secvență pe 1 bit (0, 1, 0, 1 …)
Un singur cadru este în așteptare, deci două numere ajung pentru ca receptorul să deosebească un cadru nou de unul retransmis. ACK (Acknowledgment – confirmare) n înseamnă „aștept cadrul n”.

Stop-and-wait: recuperarea după pierderi



Temporizatorul + retransmisia recuperează pierderile; bitul de secvență împiedică livrarea de două ori a unui duplicat.

Eficiența stop-and-wait: emițătorul mai mult așteaptă

Un ciclu = transmisia unui cadru + așteptarea unui drum dus-întors

T_t	RTT (Round-Trip Time – timp dus-întors) — emițătorul este inactiv, așteaptă ACK (Acknowledgment – confirmare)
L / R	2 × întârzierea de propagare

U = T_t / (T_t + RTT (Round-Trip Time – timp dus-întors))

Debit = U · R

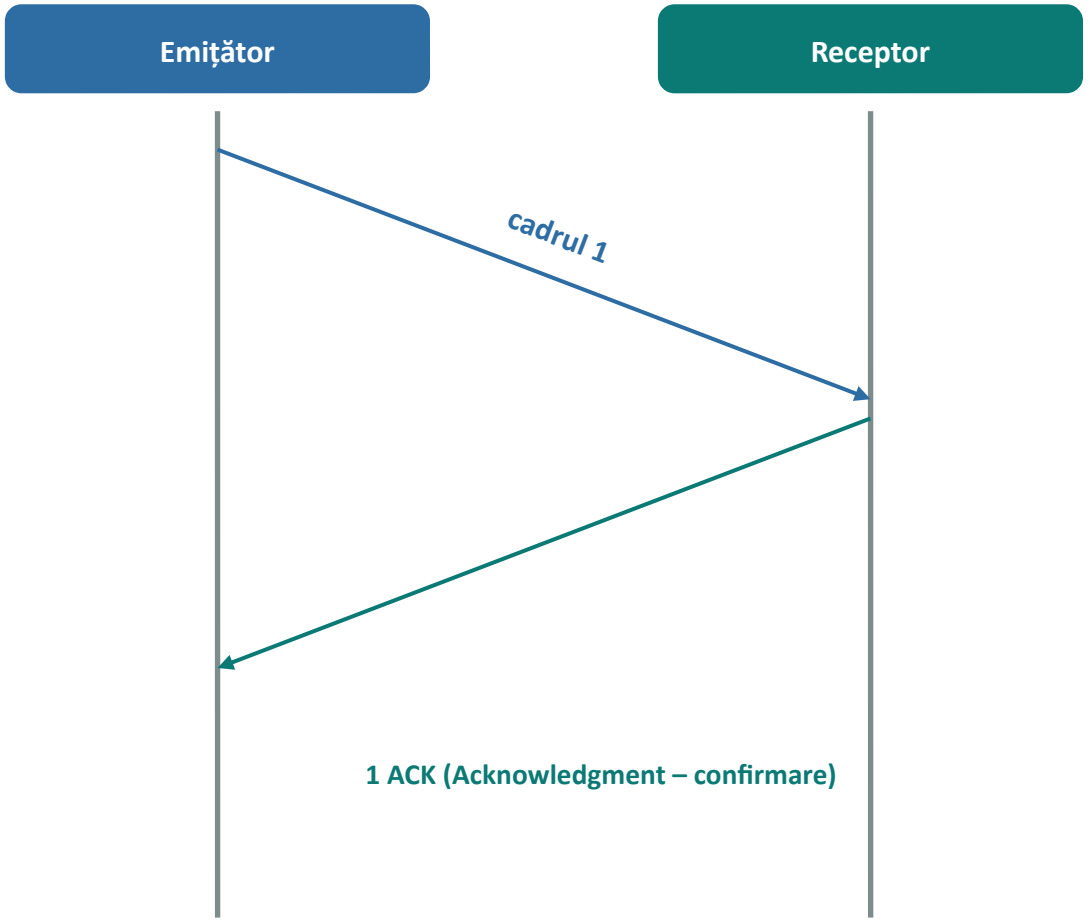
T_t = L/R (lungimea cadrului / debitul legăturii); timpul de transmisie al ACK (Acknowledgment – confirmare) este neglijat.

Două exemple rezolvate

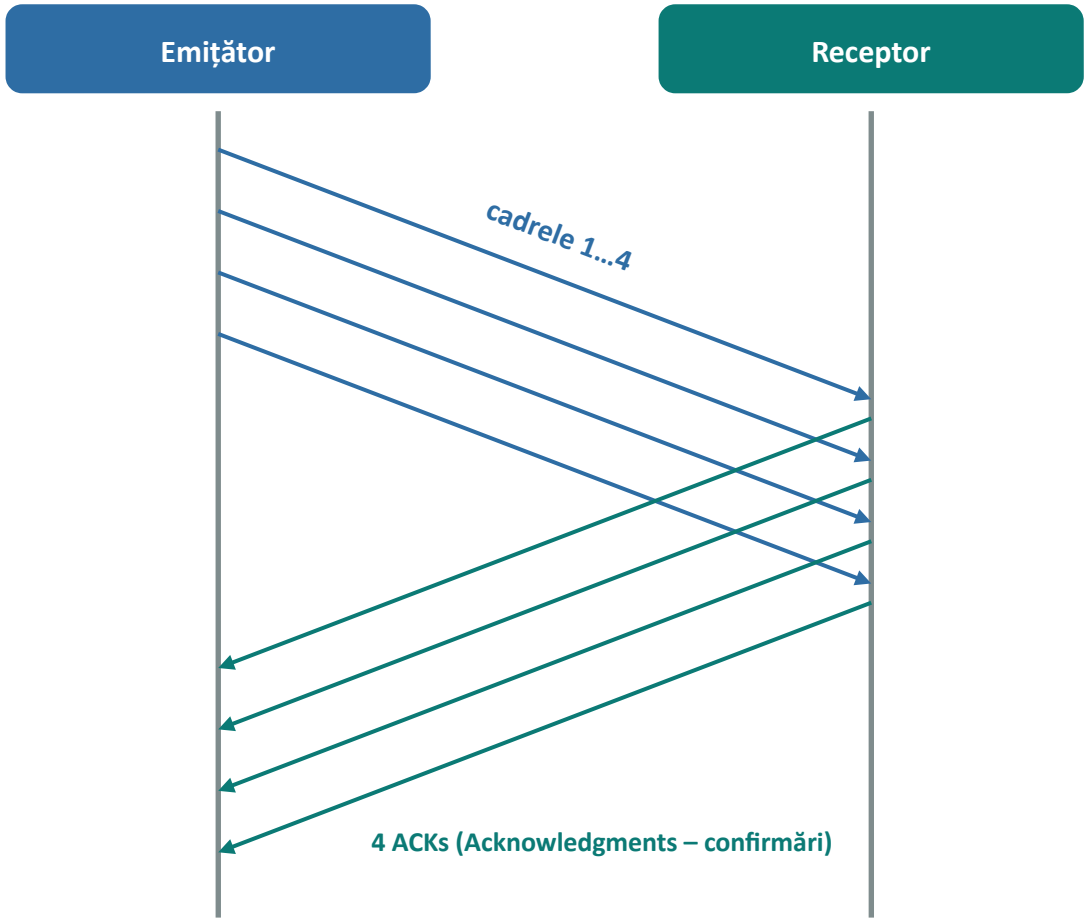
- **10 Mbit/s, 1250 B, 10 ms într-un sens**
  - T_t = 1 ms, RTT (Round-Trip Time – timp dus-întors) = 20 ms → U = 1/21 ≈ 4,8 % → 476 kbit/s
- **1 Gbit/s, 1500 B, RTT (Round-Trip Time – timp dus-întors) 30 ms**
  - T_t = 12 µs → U ≈ 0,04 % → ≈ 400 kbit/s pe o legătură de 1 Gbit/s!
- **Concluzie: legăturile lungi sau rapide necesită TRANSMISIE ÎN BANDĂ (pipelining)**

Transmisia în bandă (pipelining): mai multe cadre în tranzit

Stop-and-wait: 1 cadru la fiecare RTT (Round-Trip Time – timp dus-întors)

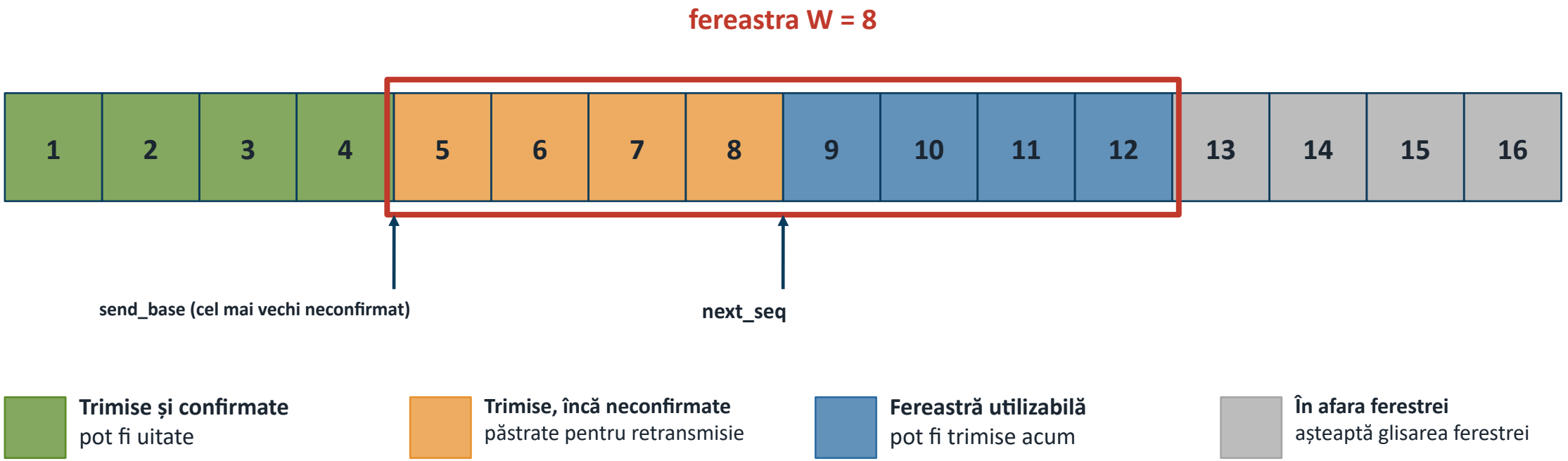


Fereastră glisantă W = 4: 4 cadre la fiecare RTT (Round-Trip Time – timp dus-întors)



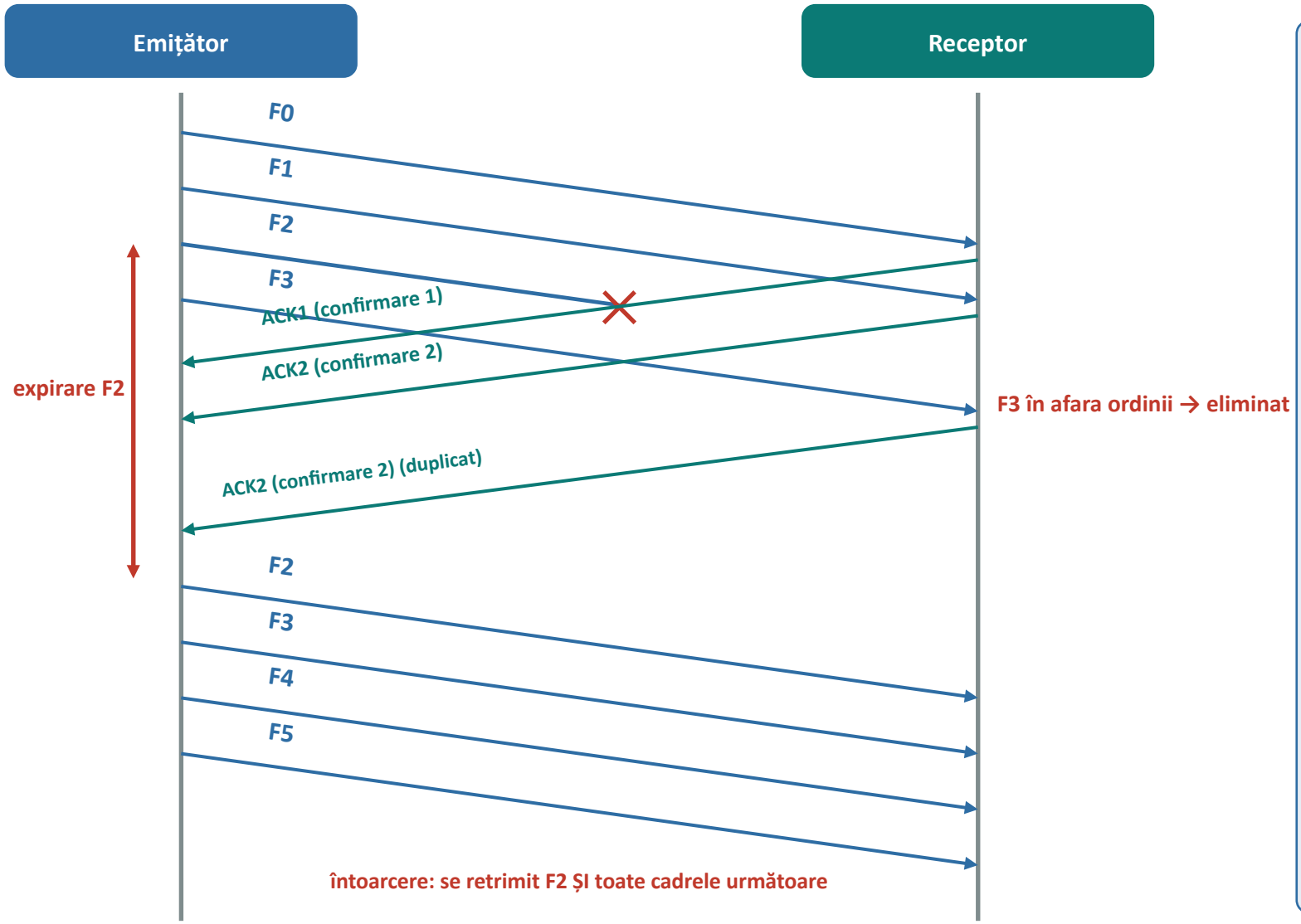
Gradul de utilizare crește de W ori (până la umplerea legăturii): U = min(1, W · T_t / (T_t + RTT (Round-Trip Time – timp dus-întors)))

Fereastra glisantă: spațiul numerelor de secvență văzut de emițător



Când sosește ACK (Acknowledgment – confirmare) pentru cadrul 5: send_base trece la 6, fereastra glisează cu o poziție spre dreapta, cadrul 13 poate fi trimis. Dimensiunea ferestrei este limitată de bufferul receptorului (controlul fluxului) și, în TCP (Transmission Control Protocol), de rețea (controlul congestiei).

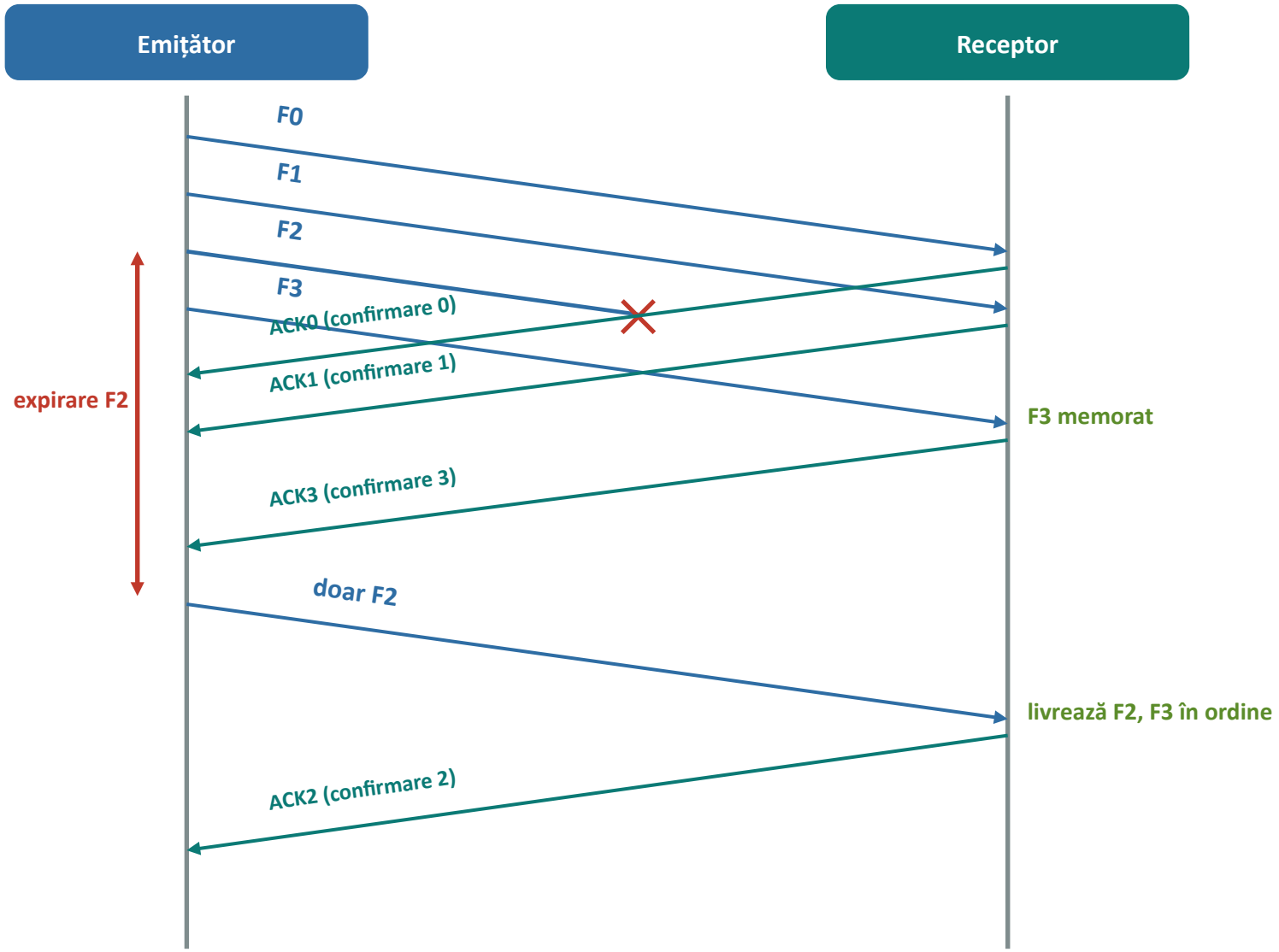
Recuperarea erorilor 1: Go-Back-N (W = 4, cadrul 2 pierdut)



Regulile Go-Back-N

- fereastra receptorului = 1: acceptă doar cadrul următor așteptat
- cadrele sosite în altă ordine sunt ELIMINATE
- ACKs (Acknowledgments – confirmări) sunt cumulative: ACK (Acknowledgment – confirmare) n = toate cadrele < n au fost primite
- un singur temporizator (pentru cel mai vechi cadru neconfirmat)
- la expirare: se retrimit acel cadru și toate cele de după el
- numere de secvență pe k biți → W ≤ 2^k − 1

Recuperarea erorilor 2: Selective Repeat (W = 4, cadrul 2 pierdut)



Regulile Selective Repeat

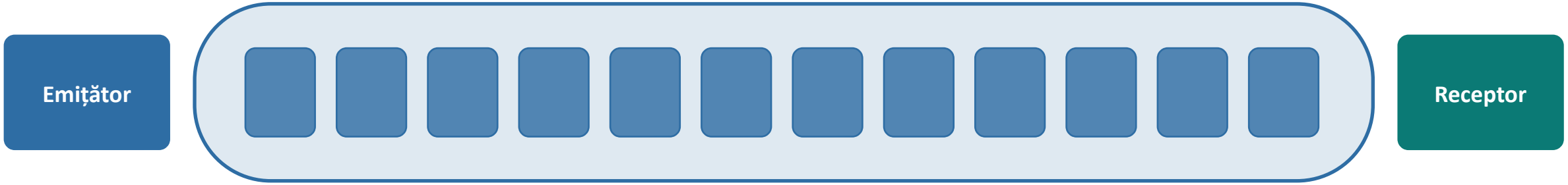
- fereastra receptorului = W: cadrele sosite în altă ordine sunt MEMORATE
- fiecare cadru este confirmat individual
- câte un temporizator pentru fiecare cadru în așteptare
- la expirare: se retrimite doar acel cadru
- livrarea către aplicație rămâne în ordine
- numere de secvență pe k biți → W ≤ 2^(k−1)

Go-Back-N comparativ cu Selective Repeat

	Go-Back-N	Selective Repeat
Fereastra receptorului	1	W
Cadre în afara ordinii	eliminate	memorate
Confirmări	cumulative	individuale
Temporizatoare	unul	câte unul pe cadru
La pierdere	retrimite cel pierdut + toate următoarele	retrimite doar cel pierdut
Fereastră maximă (k biți)	2^k − 1	2^(k−1)
Potrivit când	pierderi puține, hardware simplu	legături cu pierderi sau lungi

TCP (Transmission Control Protocol) este un hibrid: ACKs (Acknowledgments – confirmări) cumulative ca la GBN (Go-Back-N – întoarcere cu N) + memorare la receptor și SACK (Selective Acknowledgment – confirmare selectivă) ca la SR (Selective Repeat – repetare selectivă)

Dimensionarea ferestrei: produsul lățime de bandă × întârziere



„conducta” conține R × RTT (Round-Trip Time – timp dus-întors) biți — fereastra trebuie să-i acopere pe toți pentru ca legătura să fie mereu ocupată

W ≥ R · RTT (Round-Trip Time – timp dus-întors)

(în cadre: W ≥ (T_t + RTT (Round-Trip Time – timp dus-întors)) / T_t)

Aceeași legătură ca înainte: 10 Mbit/s, 1250 B, RTT (Round-Trip Time – timp dus-întors) 20 ms

- W ≥ 21 de cadre pentru 100 %; W = 8 → 38 %
- biți de secvență: GBN (Go-Back-N – întoarcere cu N) 5, SR (Selective Repeat – repetare selectivă) 6

Exemplu: 100 Mbit/s, RTT (Round-Trip Time – timp dus-întors) = 80 ms

- BDP (Bandwidth-Delay Product – produsul lățime de bandă × întârziere) = 10^8 × 0,08 = 8·10^6 biți = 1.000.000 de octeți
- fereastră simplă pe 16 biți: 65.535 B pe RTT (Round-Trip Time – timp dus-întors) → maximum 6,55 Mbit/s (6,5 % din legătură)
- Window Scale S = 4 → 65.535 × 16 ≈ 1 MB (megaoctet) → legătura poate fi umplută

Controlul fluxului în TCP (Transmission Control Protocol): fereastra de recepție (rwnd)

Bufferul receptorului (RcvBuffer)



Fiecare segment trimis de receptor conține Window = rwnd. Emițătorul respectă:
LastByteSent − LastByteAcked ≤ min(rwnd, cwnd)

Controlul fluxului ≠ controlul congestiei
rwnd protejează RECEPTORUL și este anunțată de acesta; cwnd protejează REȚEAUA și este estimată de emițător.

Cazuri speciale

- **Fereastră zero**
  - receptor plin → Window = 0; emițătorul se oprește și trimite mici sonde de fereastră până apare spațiu
- **Window Scale**
  - câmp de 16 biți × 2^S (negociat în SYN (Synchronize – sincronizare)) pentru căi cu BDP (Bandwidth-Delay Product – produsul lățime de bandă × întârziere) mare
- **Auto-reglare**
  - Linux și Windows măresc automat bufferul

Temporizatoare: estimarea RTO (Retransmission Timeout – temporizare de retransmisie) și retransmisia rapidă

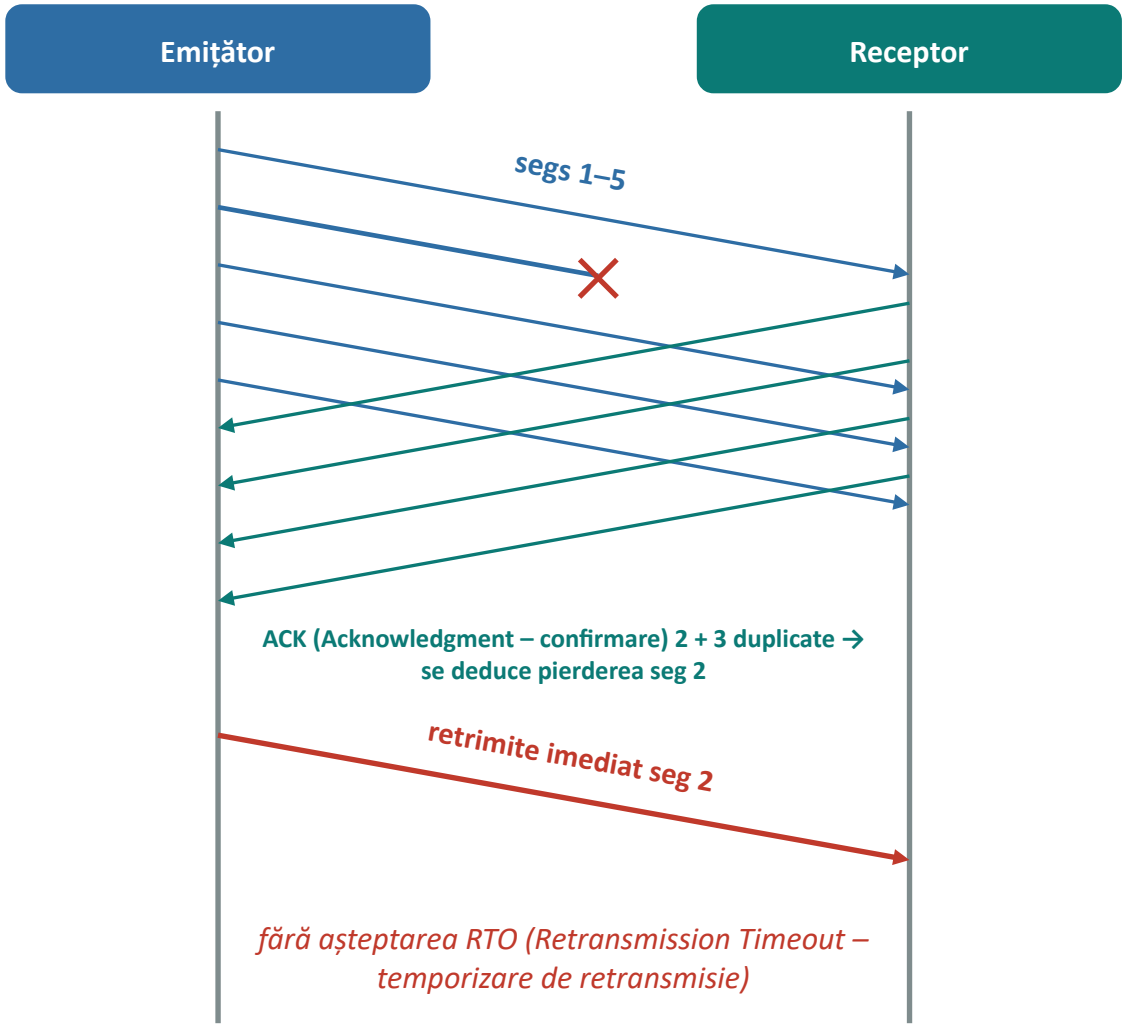
RTO (Retransmission Timeout – temporizare de retransmisie) din eșantioane RTT (Round-Trip Time – timp dus-întors) (RFC (Request for Comments – document de standardizare IETF) 6298); srtt = SRTT (Smoothed Round-Trip Time – timp dus-întors netezit), rttvar = RTTVAR (Round-Trip Time Variation – variația timpului dus-întors), rto = RTO (Retransmission Timeout – temporizare de retransmisie)

primul eșantion R	srtt = R, rttvar = R/2
eșantioanele următoare	rttvar = ¾·rttvar + ¼·|srtt − R|
	srtt = ⅞·srtt + ⅛·R
valoarea temporizării	rto = srtt + 4·rttvar (minimum 1 s; Linux 200 ms)
la expirare	rto = 2·rto (dublare exponențială)

Exemplu: eșantioane de 100 ms, apoi 140 ms

- $rto_1 = 100 + 4 \cdot 50 = 300$ ms
- rttvar = 47,5, srtt = 105 → $rto_2 = 295$ ms

Retransmisia rapidă: 3 ACKs (Acknowledgments – confirmări) duplicate

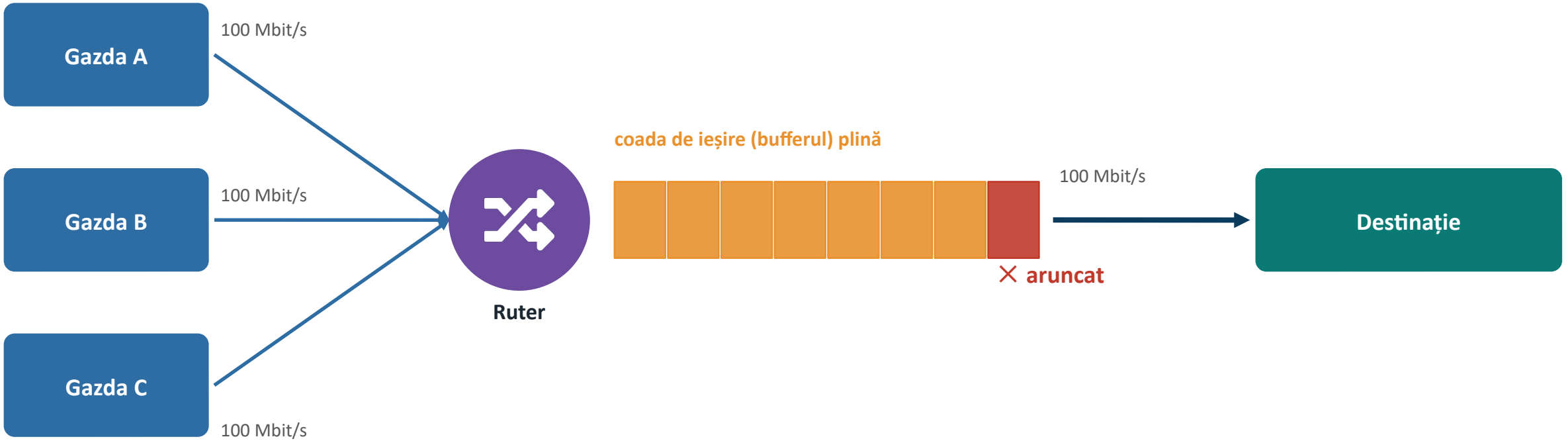


7

Congestia traficului și controlul congestiei în TCP (Transmission Control Protocol)

Prea multe surse, prea rapide, pentru prea puțină capacitate

Ce este congestia? Cozile se umplu, pachetele se pierd



300 Mbit/s oferiți > 100 Mbit/s capacitate → CONGESTIE

cozi mai lungi
cresc întârzierea și jitter-ul

→

depășirea bufferului
pachete aruncate

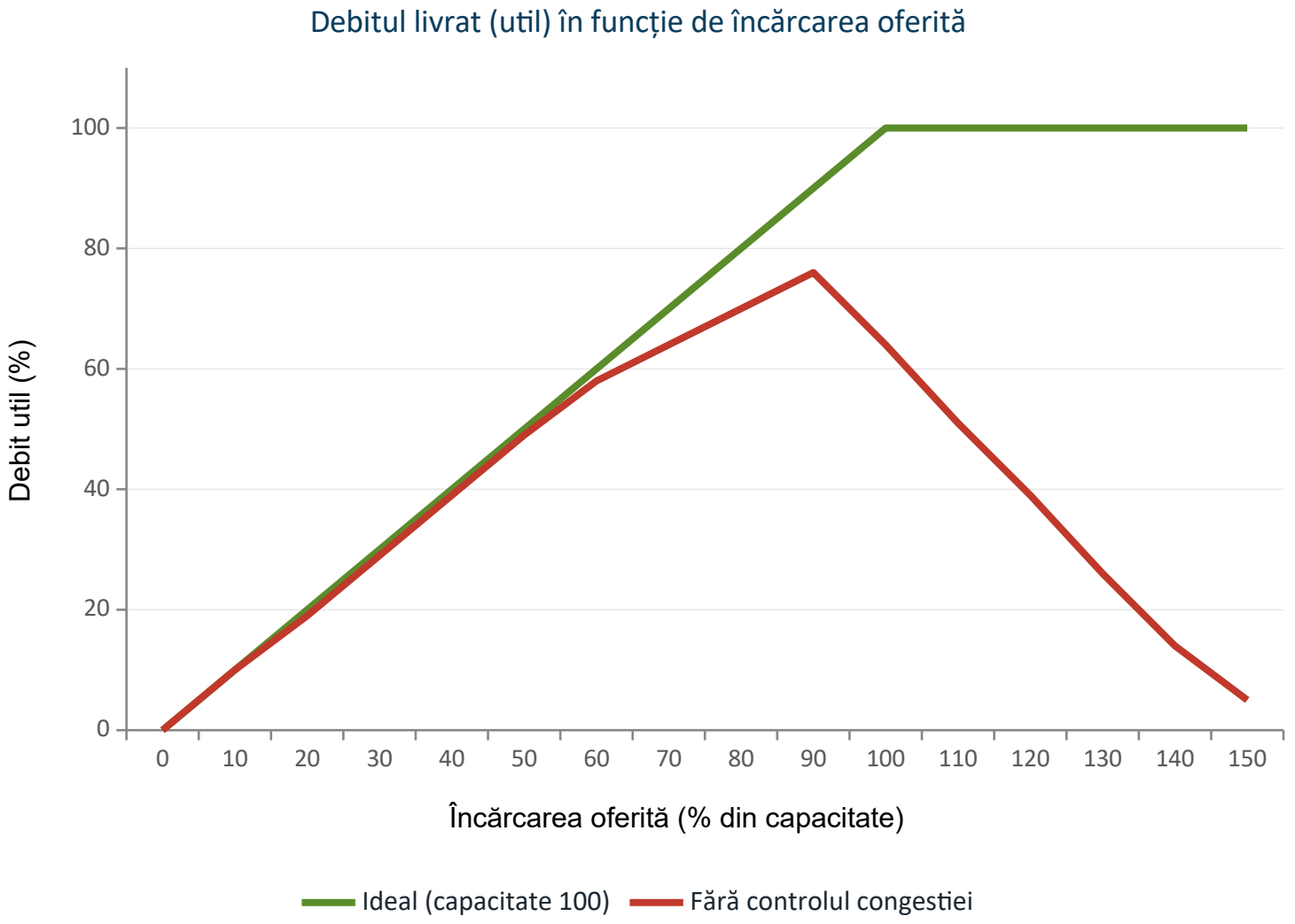
→

retransmisii
și mai multă încărcare

→

colaps de congestie
debit util foarte mic

Colapsul de congestie: mai multă încărcare oferită, mai puțin debit util



Cum citim graficul

- încărcare mică: tot ce se oferă se livrează
- aproape de capacitate: cozile cresc, întârzierea crește
- suprasarcină: pierderi → retransmisii → capacitate irosită pe pachete aruncate mai târziu
- 1986: debitul Internetului a scăzut de ~1000× → controlul congestiei TCP (Transmission Control Protocol) al lui Van Jacobson (1988)

Controlul fluxului comparativ cu controlul congestiei

Controlul fluxului

- protejează bufferul RECEPTORULUI
- variabila: rwnd (fereastra de recepție)
- calculată și anunțată de receptor în fiecare segment
- informație cap-la-cap, exactă

Controlul congestiei

- protejează REȚEAUA (cozile și legăturile ruterelor)
- variabila: cwnd (fereastra de congestie)
- estimată de emițător din pierderi, ACKs (Acknowledgments – confirmări) duplicate, întârziere, marcaje ECN (Explicit Congestion Notification – notificare explicită a congestiei)
- informație dedusă, aproximativă

Fereastra efectivă de emisie

$$W = \min(rwnd, cwnd)$$

Abordări ale controlului congestiei

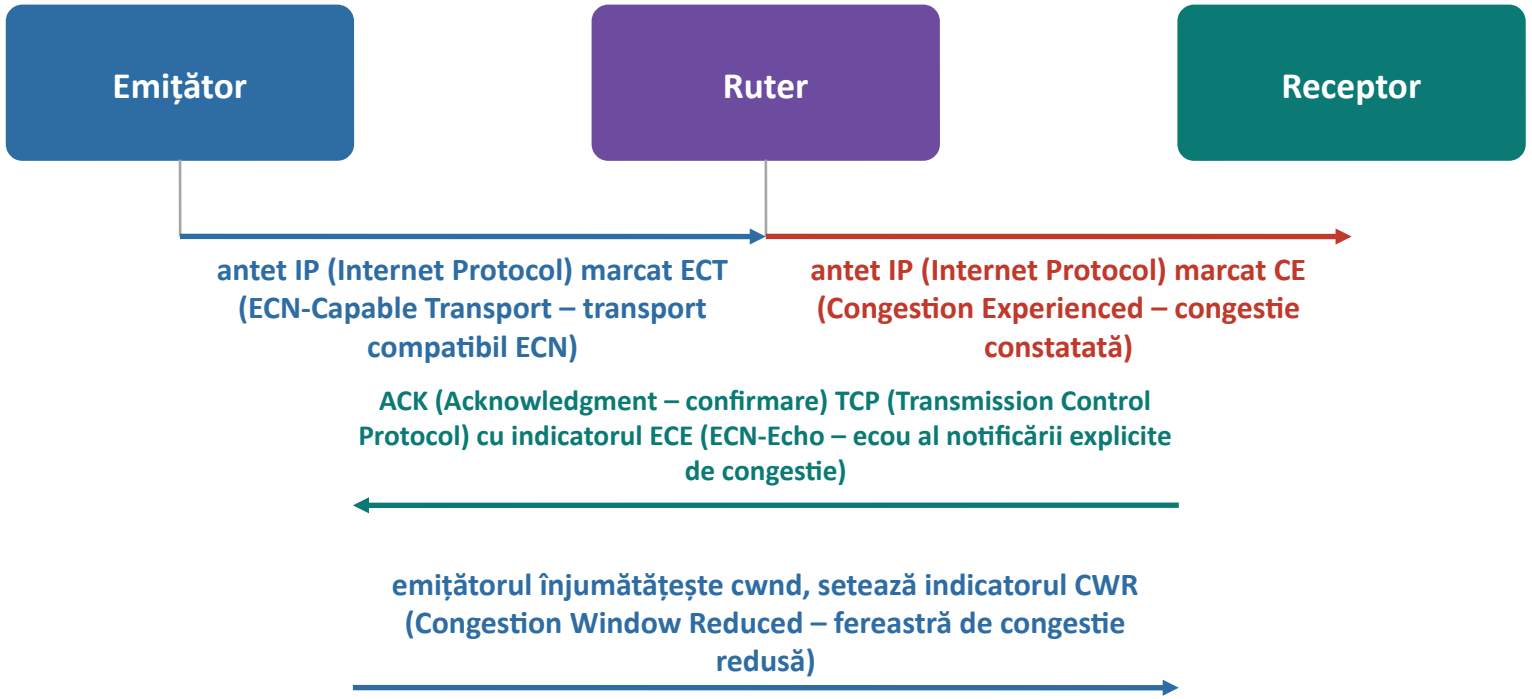
Cap-la-cap (TCP (Transmission Control Protocol) clasic)

- rețeaua nu dă niciun semnal explicit
- emițătorii deduc congestia din pierderi și întârziere
- Reno, CUBIC, BBR (Bottleneck Bandwidth and Round-trip propagation time)

Gestiunea cozilor în rutere

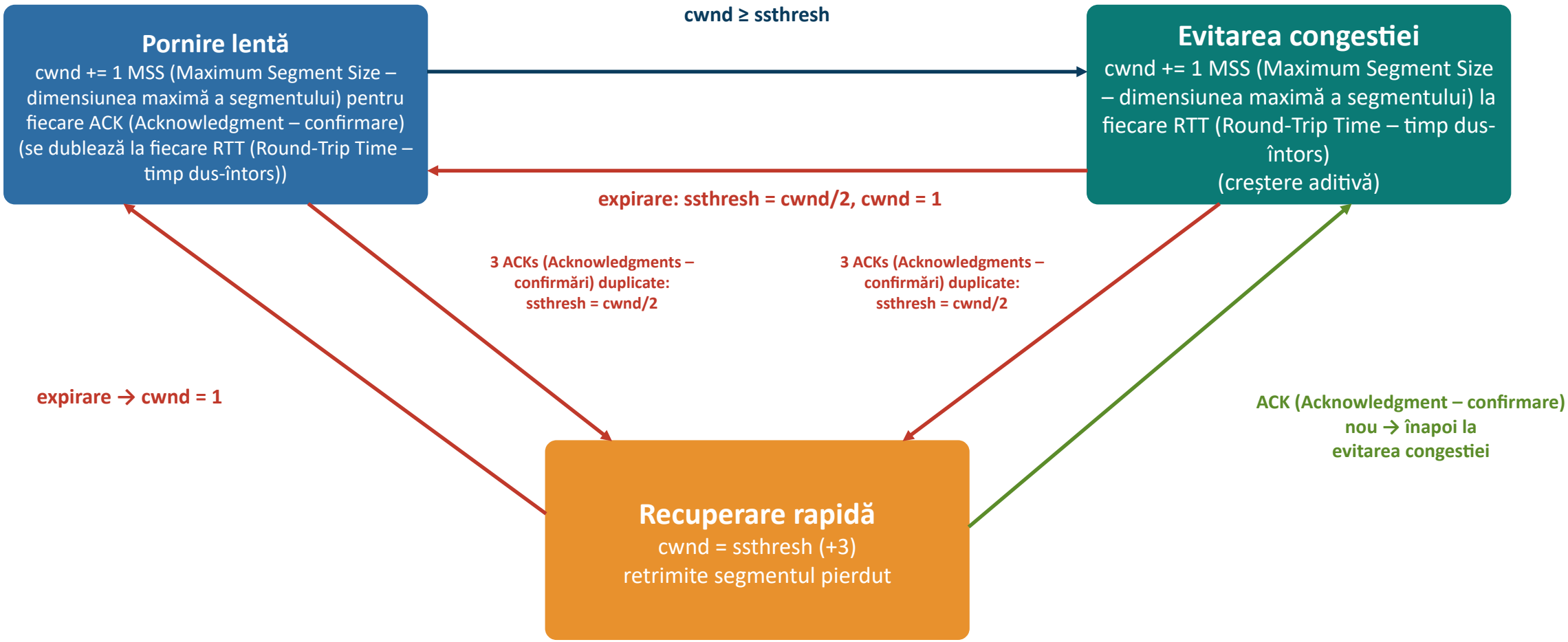
- AQM (Active Queue Management – gestiune activă a cozilor): RED (Random Early Detection – detecție timpurie aleatoare), CoDel (Controlled Delay – întârziere controlată), fq_codel
- aruncă sau marchează DIN TIMP pentru a menține cozile scurte
- evită bufferbloat

Asistată de rețea: ECN (Explicit Congestion Notification – notificare explicită a congestiei) (RFC (Request for Comments – document de standardizare IETF) 3168)



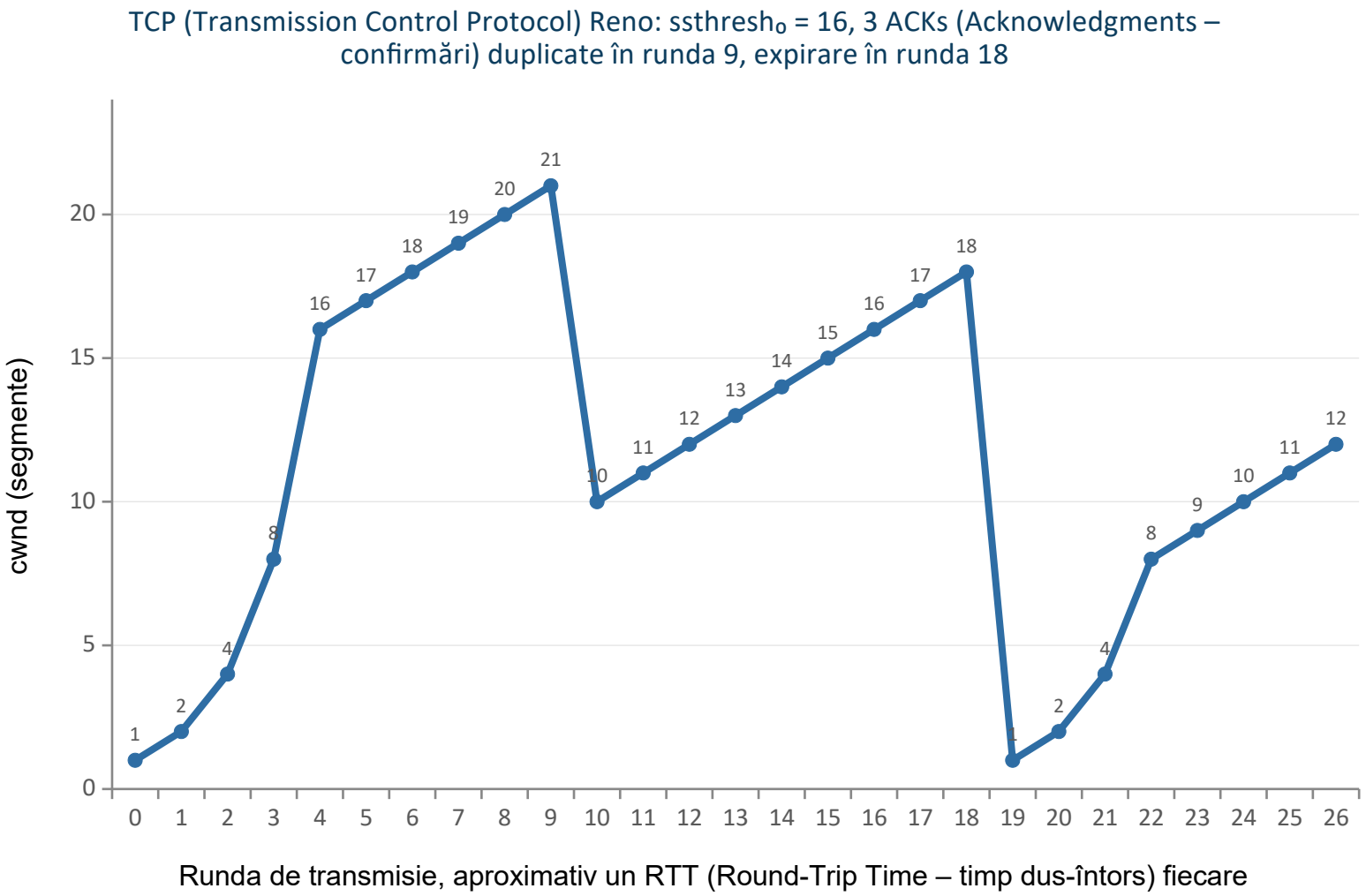
Avantaj: congestia este semnalată FĂRĂ aruncarea pachetelor — nu e nevoie de retransmisie

Controlul congestiei în TCP (Transmission Control Protocol): automatul de stări (Reno)



Start: cwnd = 1–10 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) (RFC (Request for Comments – document de standardizare IETF) 6928 permite 10), ssthresh = mare. TCP (Transmission Control Protocol) Tahoe nu are recuperare rapidă: orice pierdere → cwnd = 1.

Evoluția cwnd: dinții de fierăstrău AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă)



Fazele din grafic

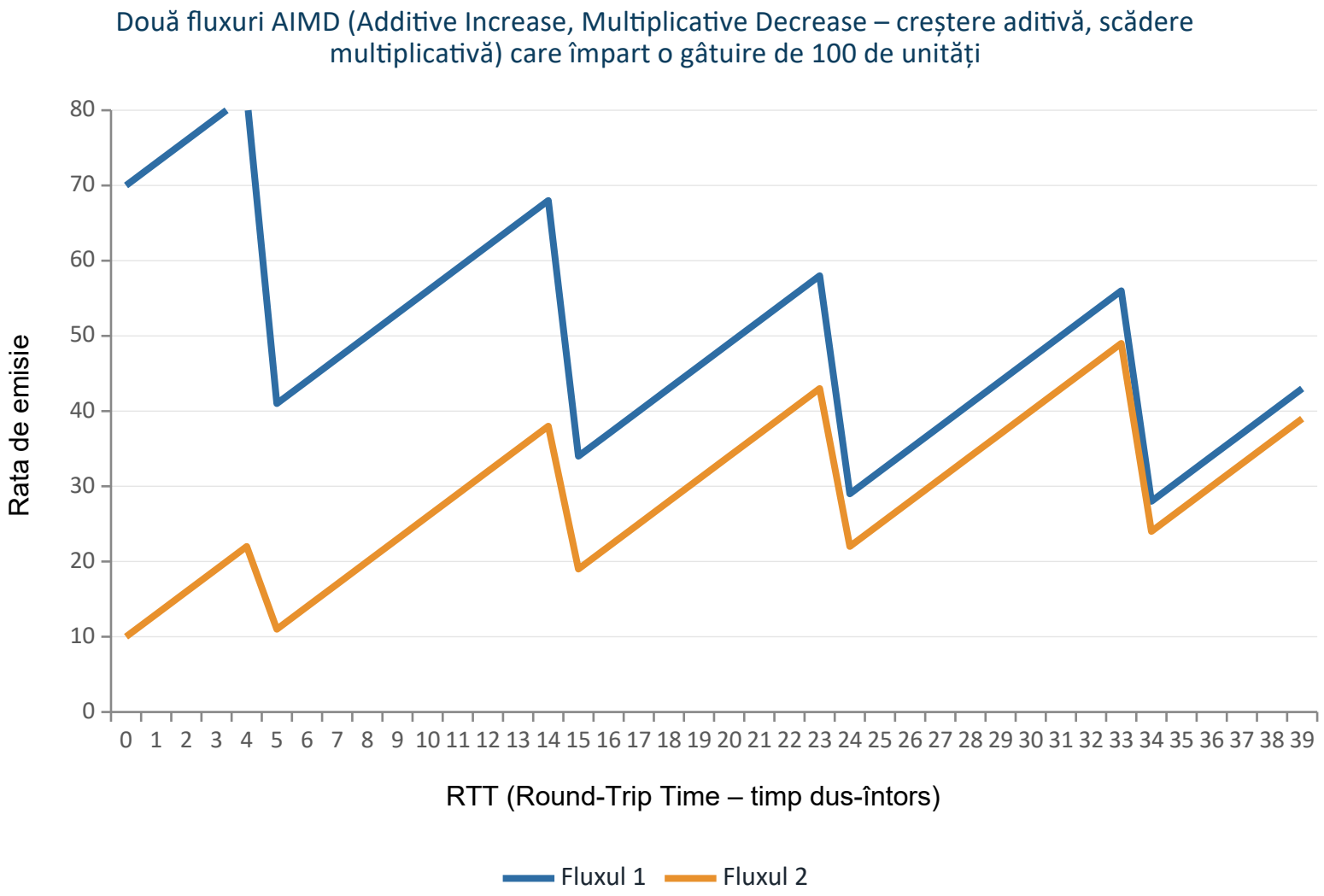
- rundele 0–4: pornire lentă 1→2→4→8→16
- rundele 4–9: evitarea congestiei 16→21
- runda 9, 3 ACKs (Acknowledgments – confirmări) duplicate: ssthresh = 10, cwnd = 10
- rundele 10–18: creștere liniară 10→18
- runda 18, expirare: ssthresh = 9, cwnd = 1
- rundele 19–22: pornire lentă 1→8, apoi limitare la 9

Regulile într-un singur tabel

Eveniment / fază	Regulă	Efect
Pornire lentă (cwnd < ssthresh)	pentru fiecare ACK (Acknowledgment – confirmare): cwnd += 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului)	exponențial: ×2 la fiecare RTT (Round-Trip Time – timp dus-întors)
Evitarea congestiei (cwnd ≥ ssthresh)	pentru fiecare ACK (Acknowledgment – confirmare): cwnd += MSS (Maximum Segment Size – dimensiunea maximă a segmentului)²/cwnd	liniar: +1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) la fiecare RTT (Round-Trip Time – timp dus-întors)
3 ACKs (Acknowledgments – confirmări) duplicate (Reno)	ssthresh = cwnd/2; cwnd = ssthresh (+3); retransmisie rapidă	scădere multiplicativă, moderată
Expirarea temporizatorului de retransmisie	ssthresh = cwnd/2; cwnd = 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului)	repornire cu pornire lentă, severă
Ecou ECN (Explicit Congestion Notification – notificare explicită a congestiei) (ECE (ECN-Echo – ecou al notificării explicite de congestie))	reacție ca la o pierdere, o dată la fiecare RTT (Round-Trip Time – timp dus-întors)	scădere fără nicio pierdere

AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă): creștere cu un segment la fiecare drum dus-întors, înjumătățire la pierdere.

AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă) asigură echitatea: două fluxuri converg spre o cotă egală



De ce converg

- ambele fluxuri adaugă ACEEAȘI cantitate pe RTT (Round-Trip Time – timp dus-întors) → diferența rămâne
- la pierdere ambele se ÎNJUMĂTĂȚESC → diferența se înjumătățește
- după câteva cicluri ambele oscilează în jurul lui 100/2
- N fluxuri → fiecare ≈ R/N
- atenție: multe conexiuni paralele sau UDP (User Datagram Protocol) fără control al congestiei strică echitatea

Algoritmi de control al congestiei în practică

Tahoe (1988)

- pierdere → cwnd = 1, pornire lentă; fără recuperare rapidă
- **istoric**

Reno / NewReno

- retransmisie rapidă + recuperare rapidă; AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă)
- **algoritm de referință (RFC (Request for Comments – document de standardizare IETF) 5681)**

CUBIC

- cwnd crește ca funcție cubică de timpul de la ultima pierdere; rapid pe căi cu BDP (Bandwidth-Delay Product – produsul lățime de bandă × întârziere) mare
- **implicit: Linux ≥ 2.6.19, Windows 10 1709+, macOS**

Compound TCP (Transmission Control Protocol)

- fereastră bazată pe pierderi + fereastră bazată pe întârziere, combinate
- **algoritmul implicit mai vechi din Windows**

BBR (Bottleneck Bandwidth and Round-trip propagation time) (Google)

- bazat pe model: măsoară lățimea de bandă a gâtuirii și RTT (Round-Trip Time – timp dus-întors) minim, dozează pachetele
- **modulul Linux tcp_bbr; YouTube, Google Cloud**

Implementări QUIC (Quick UDP Internet Connections)

- controlul congestiei în spațiul utilizator (CUBIC sau BBR (Bottleneck Bandwidth and Round-trip propagation time))
- **HTTP/3 (Hypertext Transfer Protocol versiunea 3)**

```
Verificare:  Linux → sysctl net.ipv4.tcp_congestion_control
        Windows → Get-NetTCPSetting | Select SettingName, CongestionProvider
```

Rezumat: nivelul transport pe o singură pagină

